

Sustainable Logistics Project for La Nébuleuse: VRP and FLP Optimization

VRP

```
str: url_distances

!pip install ortools
!pip install folium, 'https://raw.githubusercontent.com/amathie5/...'

import pandas as pd
import numpy as np
import folium
import math
import requests
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import io
import os
from google.colab import files
from ortools.linear_solver import pywraplp
from ortools.constraint_solver import routing_enums_pb2, pywrapcp
```

```
Collecting ortools
  Downloading ortools-9.15.6755-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata
Collecting absl-py>=2.0.0 (from ortools)
  Downloading absl_py-2.4.0-py3-none-any.whl.metadata (3.3 kB)
Requirement already satisfied: numpy>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from ortools)
Requirement already satisfied: pandas>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from ortools)
Collecting protobuf<6.34,>=6.33.1 (from ortools)
  Downloading protobuf-6.33.6-cp39-abi3-manylinux2014_x86_64.whl.metadata (593 bytes)
Requirement already satisfied: typing-extensions>=4.12 in /usr/local/lib/python3.12/dist-packages (from ortools)
Requirement already satisfied: immutabledict>=3.0.0 in /usr/local/lib/python3.12/dist-packages (from ortools)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from ortools)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=2.0.0)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=2.0.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil)
Downloading ortools-9.15.6755-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (29.8 MB)
  https://raw.githubusercontent.com/amathie5/La-Nebuleuse/refs/heads/main/ortools-9.15.6755-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (29.8 MB)
  29.8/29.8 MB 22.5 MB/s eta 0:00:00
Downloading absl_py-2.4.0-py3-none-any.whl (135 kB)
  135.8/135.8 kB 8.3 MB/s eta 0:00:00
Downloading protobuf-6.33.6-cp39-abi3-manylinux2014_x86_64.whl (323 kB)
  323.4/323.4 kB 12.9 MB/s eta 0:00:00
Installing collected packages: protobuf, absl-py, ortools
  Attempting uninstall: protobuf
    Found existing installation: protobuf 5.29.6
    Uninstalling protobuf-5.29.6:
      Successfully uninstalled protobuf-5.29.6
  Attempting uninstall: absl-py
    Found existing installation: absl-py 1.4.0
    Uninstalling absl-py-1.4.0:
      Successfully uninstalled absl-py-1.4.0
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed
grpcio-status 1.71.2 requires protobuf<6.0dev,>=5.26.1, but you have protobuf 6.33.6 which is incompatible
google-ai-generativelanguage 0.6.15 requires protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5
Successfully installed absl-py-2.4.0 ortools-9.15.6755 protobuf-6.33.6
WARNING: The following packages were previously imported in this runtime:
[google]
You must restart the runtime in order to use newly installed versions.
```

RESTART SESSION

```
Requirement already satisfied: folium in /usr/local/lib/python3.12/dist-packages (0.20.0)
Requirement already satisfied: branca>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from folium) (0.8.2)
Requirement already satisfied: Jinja2>=2.9 in /usr/local/lib/python3.12/dist-packages (from folium) (3.1.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from folium) (2.0.2)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from folium) (2.32.0)
Requirement already satisfied: xyzservices in /usr/local/lib/python3.12/dist-packages (from folium) (2024.10.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2) (3.0.2)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests) (2025.1.31)
```

```
# 1. Définir les URLs "Raw" de tes fichiers GitHub
# Remplace ces liens par les vrais liens Raw de ton repository
url_donnees = "https://raw.githubusercontent.com/amathie5/La-Nebuleuse/refs/heads/main/donn%C3%A9es2022.csv"
url_packaging = "https://raw.githubusercontent.com/amathie5/La-Nebuleuse/refs/heads/main/packaging.csv"
url_distances = "https://raw.githubusercontent.com/amathie5/La-Nebuleuse/refs/heads/main/matrice_distances.csv"

# 2. Lire les fichiers directement dans des DataFrames Pandas
# Note: si tes CSV ont été générés avec un Excel français, le séparateur est souvent un point-virgule.
# Si c'est le cas, ajoute l'argument sep=";" comme ceci : pd.read_csv(url_donnees, sep=";")
df_sales = pd.read_csv(url_donnees, sep=";", decimal = ",")
df_packaging = pd.read_csv(url_packaging, sep=";", decimal = ",")
df_distances = pd.read_csv(url_distances, index_col=0, sep=";", decimal=",")

# 3. Afficher le DataFrame (cela générera un tableau propre comme sur ta capture d'écran)
display(df_sales.head())
display(df_packaging.head())
```

	COMPANYNAME	Customer__No__	Customer_Name	LiterCaption	ValueEntr	
str: url_distances View 'https://raw.githubusercontent.com/amathie5/20544258974c267d37f17c1409612784/sl_project_vf-ev.ipynb#scrollTo=32bSGrTX2kFY&printM...	La Nebuleuse SA	1009	Great Escape	Liter		
	La Nebuleuse SA	1009	Great Escape	Liter		
	La Nebuleuse SA	1009	Great Escape	Liter		
	La Nebuleuse SA	1010	Les Docks	Liter		
	La Nebuleuse SA	1010	Les Docks	Liter		
	Batch	DLC	Recettes Contract	Batch.1	Quel Type de Conditionnement avons-nous utilisé ?	Production Totale
3	159.0	31.01.2026	NaN	NaN	Can	10513
/	159.0	06.01.2026	NaN	NaN	Cuv	1000
4	26.0	31.01.2026	NaN	NaN	Bot	5592
3	26.0	31.01.2026	NaN	NaN	Bot	6648
4	202.0	31.01.2026	NaN	NaN	Bot	552

```
# Data cleaning
cols_to_keep = [
    "Customer_Name",
    "Item_Description",
    "Item_Base_Unit_of_Measure_",
    "ValueEntryBuffer__Invoiced_Quantity_"
]
df_clean = df_sales[cols_to_keep].copy()
df_clean.columns = ["Customer", "Description", "Unit", "Quantity"]
df_clean["Quantity"] = df_clean["Quantity"].astype(str)
df_clean["Quantity"] = df_clean["Quantity"].str.replace(", ", "", regex=False)
df_clean["Quantity"] = pd.to_numeric(df_clean["Quantity"], errors="coerce")

# Filter out returns and deposits ("Caution")
df_clean = df_clean[~df_clean["Description"].str.contains("Caution", case=False, na=False)]
df_clean = df_clean[df_clean["Quantity"] > 0]

# Categorize items
def classify_item(unit):
    if pd.isna(unit):
        return "Other"
    unit = str(unit).upper()
    if unit == "F":
        return "Keg"
    if unit in ["6X4B", "24B"]:
        return "Bottle_Carton"
    if unit in ["6X4C", "24C", "4X6C"]:
        return "Can_Carton"
    if unit == "L":
        return "Bulk_Liters"
```

```

return "Other"

df_clean["Category"] = df_clean["Unit"].apply(classify_item)

# Calculate Pallets and Weights
def calc_logistics(row):
    cat = row["Category"]
    qty = row["Quantity"]
    weight = 0.0
    pallets = 0.0

    if cat == "Keg":
        url_distances = str: url_distances
        pallets = qty / 30.0
        weight = (qty * 26.0) + (pallets * 40.0)
    elif cat == "Bottle_Carton":
        pallets = qty / 54.0
        weight = (qty * 14.0) + (pallets * 20.0)
    elif cat == "Can_Carton":
        pallets = qty / 63.0
        weight = (qty * 12.0) + (pallets * 20.0)
    elif cat == "Bulk_Liters":
        weight = qty * 1.0 # 1 liter = ~ 1 kg
        pallets = 0.0 # Bulk goes in the tanker, no pallets

    return pd.Series([weight, pallets])

df_clean[["Total_Weight_kg", "Total_Pallets"]] = df_clean.apply(calc_logistics, axis=1)

# Aggregate data per customer for the whole year 2025
df_customer_yearly = df_clean.groupby("Customer").agg(
    Yearly_Weight_kg=("Total_Weight_kg", "sum"),
    Yearly_Pallets=("Total_Pallets", "sum"),
    Yearly_Bulk_Liters=("Quantity", lambda x: x[df_clean.loc[x.index, "Category"] == "Bulk_Liters"].
).reset_index()

# Calculate Weekly Averages (assuming 52 weeks)
df_customer_weekly = df_customer_yearly.copy()
df_customer_weekly["Weekly_Weight_kg"] = df_customer_yearly["Yearly_Weight_kg"] / 52.0
df_customer_weekly["Weekly_Pallets"] = df_customer_yearly["Yearly_Pallets"] / 52.0
df_customer_weekly["Weekly_Bulk_Liters"] = df_customer_yearly["Yearly_Bulk_Liters"] / 52.0

display(df_customer_weekly.head(10))

```

	Customer	Yearly_Weight_kg	Yearly_Pallets	Yearly_Bulk_Liters	Weekly_Weight_kg	Weekly_Pallets
0	AlloBoissons SA	72518.476190	91.890476	0.0	1394.586081	1.767125
1	Doki Doki - Mukmuk Sàrl	6668.740741	8.170370	0.0	128.245014	0.157123
2	Great Escape	11206.666667	13.666667	0.0	215.512821	0.262821
3	Hospitality Partners Sàrl (Le Pointu)	str: url_distances View 182.000000 'https://raw.githubusercontent.com/amathi...'	5.100000	0.0	80.423077	0.098077
4	Jetée de la compagnie	527.777778	0.655556	0.0	10.149573	0.012607
5	La Rincette Sàrl	2186.666667	2.666667	0.0	42.051282	0.051282
6	Les Docks	15180.296296	18.681481	0.0	291.928775	0.359259
7	Loxton	2587.925926	3.162963	0.0	49.767806	0.060826
8	MKK SARL (Simplon)	3362.000000	4.100000	0.0	64.653846	0.078846
9	Migros Partenaire Ouchy Sàrl	4079.132275	5.256614	0.0	78.444851	0.101089

```
# Definition of mapping for operational weeks per client
# 52 weeks, except newer clients
weeks_mapping = {
    "Simplon": 12.0,
    "La Latina": 26.0
}

# Injection of missing data for La Latina manually
la_latina_data = pd.DataFrame([
    {"Customer": "La Latina", "Description": "Manual Entry Keg", "Unit": "F", "Quantity": 75},
    {"Customer": "La Latina", "Description": "Manual Entry Bottles", "Unit": "24B", "Quantity": 3}
])

df_clean = pd.concat([df_clean, la_latina_data], ignore_index=True)

# 3. Recalculate Logistics with the newly added data
df_clean["Category"] = df_clean["Unit"].apply(classify_item)
df_clean[["Total_Weight_kg", "Total_Pallets"]] = df_clean.apply(calc_logistics, axis=1)

# 4. Aggregate Yearly Data
df_yearly = df_clean.groupby("Customer").agg(
    Yearly_Weight_kg=("Total_Weight_kg", "sum"),
    Yearly_Pallets=("Total_Pallets", "sum"),
    Yearly_Bulk_Liters=("Quantity", lambda x: x[df_clean.loc[x.index, "Category"] == "Bulk_Liters"].
).reset_index()

# 5. Apply the specific week division
def get_weeks(customer_name):
    for key, weeks in weeks_mapping.items():
        if key.lower() in customer_name.lower():
            return weeks
    return 52.0

df_yearly["Operational_Weeks"] = df_yearly["Customer"].apply(get_weeks)

# 6. Calculate True Weekly Averages
```

```
df_weekly = df_yearly.copy()
df_weekly["Weekly_Weight_kg"] = (df_weekly["Yearly_Weight_kg"] / df_weekly["Operational_Weeks"]).round(2)
df_weekly["Weekly_Pallets"] = (df_weekly["Yearly_Pallets"] / df_weekly["Operational_Weeks"]).round(2)
df_weekly["Weekly_Bulk_Liters"] = (df_weekly["Yearly_Bulk_Liters"] / df_weekly["Operational_Weeks"])

print("--- Final Weekly Logistics Demand ---")
display(df_weekly[["Customer", "Operational_Weeks", "Weekly_Weight_kg", "Weekly_Pallets"]])
```

--- Final Weekly Logistics Demand ---

	Customer	Operational_Weeks	Weekly_Weight_kg	Weekly_Pallets
0	Alloboissons SA	52.0	1394.59	1.77
1	Doki Dock - Mukmuk Sàrl	52.0	128.25	0.16
2	Great Escape	52.0	215.51	0.26
3	Hospitality Partners Sàrl (Le Pointu)	52.0	80.42	0.10
4	Jetée de la compagnie	52.0	10.15	0.01
5	La Latina	26.0	80.50	0.10
6	La Rincette Sàrl	52.0	42.05	0.05
7	Les Docks	52.0	291.93	0.36
8	Loxton	52.0	49.77	0.06
9	MKK SARL (Simplon)	12.0	280.17	0.34
10	Migros Partenaire Ouchy Sàrl	52.0	78.44	0.10
11	The Lacustre	52.0	169.50	0.21

```
# VRP Dataset (except Bulk)
df_vrp_standard = df_weekly[["Customer", "Operational_Weeks", "Weekly_Weight_kg", "Weekly_Pallets"]]

# Filter out customers who only ordered bulk
df_vrp_standard = df_vrp_standard[df_vrp_standard["Weekly_Weight_kg"] > 0].reset_index(drop=True)

print("VRP Demand (Packaged Goods)")
display(df_vrp_standard)

# Bulk Dataset
df_vrp_bulk = df_weekly[["Customer", "Operational_Weeks", "Weekly_Bulk_Liters"]].copy()

# Filter out customers who did not order bulk
df_vrp_bulk = df_vrp_bulk[df_vrp_bulk["Weekly_Bulk_Liters"] > 0].reset_index(drop=True)

print("\nBulk Flow Demand (Tank Deliveries)")
display(df_vrp_bulk)
```

VRP Demand (Packaged Goods)

	Customer	Operational_Weeks	Weekly_Weight_kg	Weekly_Pallets	
0	AlloBoissons SA	52.0	1394.59	1.77	
1	Doki Doki - Mukmuk Sàrl	52.0	128.25	0.16	
2	Great Escape	52.0	215.51	0.26	
3	Hospitality Partners Sàrl (Le Pointu)	52.0	80.42	0.10	
4	Jetée de la compagnie str: url_distances	52.0	10.15	0.01	
5	View La Latina	26.0	80.50	0.10	
6	La Rincette Sàrl https://raw.githubusercontent.com/amathi...	52.0	42.05	0.05	
7	Les Docks	52.0	291.93	0.36	
8	Loxton	52.0	49.77	0.06	
9	MKK SARL (Simplon)	12.0	280.17	0.34	
10	Migros Partenaire Ouchy Sàrl	52.0	78.44	0.10	
11	The Lacustre	52.0	169.50	0.21	

Bulk Flow Demand (Tank Deliveries)

Customer	Operational_Weeks	Weekly_Bulk_Liters	
----------	-------------------	--------------------	---

Étapes suivantes :

[Générer du code avec df_vrp_standard](#)

[New interactive sheet](#)

```
# data
fleet_data = {
    "Vehicle": ["VW_Caddy", "Ford_Transit", "Truck"],
    "Max_Weight_kg": [700, 1200, 8300],
    "Max_Pallets": [999, 999, 15], # 999 acts as infinity for smaller vehicles where weight is the n
    "Cost_per_km": [0.60, 0.80, 1.50]
}
df_fleet = pd.DataFrame(fleet_data)
display(df_fleet)

display(df_distances.iloc[:15, :15])
```

	Vehicle	Max_Weight_kg	Max_Pallets	Cost_per_km								
0	VW_Caddy	700	999	0.6								
1	Ford_Transit	1200	999	0.8								
2	Truck	8300	15	1.5								
		Brasserie Renens	Entrepôt Crissier	Great Escape	The Lacustre	Simplon	Pointu	Loxton	La Latina	Docks	Doki Doki	Jetée de la compagnie
Origine \ Destination		str: url_distances View										
Brasserie Renens		0.00	3.37	4.69	5.42	4.80	3.71	4.16	4.07	2.88	4.52	
Entrepôt Crissier		3.48	0.00	7.17	8.78	7.39	6.19	6.64	6.55	5.77	7.00	
Great Escape		4.99	9.22	0.00	2.59	1.75	1.48	0.87	0.96	2.70	0.69	
The Lacustre		4.85	7.91	2.71	0.00	1.30	3.30	2.39	2.92	3.28	2.34	
Simplon		4.73	8.31	1.87	1.30	0.00	2.46	1.54	2.08	2.43	1.50	
Pointu		4.02	8.77	0.83	3.65	2.81	0.00	1.75	1.85	2.25	1.52	
Loxton		4.12	8.35	1.48	3.05	2.20	1.74	0.00	0.09	1.82	1.58	
La Latina		4.03	8.25	1.57	2.95	2.11	1.65	0.09	0.00	1.73	1.67	
Docks		3.23	7.02	2.61	3.01	2.47	1.85	1.83	1.74	0.00	2.44	
Doki Doki		4.73	8.96	0.69	2.23	1.39	2.08	0.88	0.84	2.44	0.00	
Jetée de la compagnie		5.50	8.60	3.80	1.10	1.60	3.70	3.20	3.10	2.70	3.20	
La rincette		4.50	7.60	3.60	2.40	2.70	3.50	3.00	2.90	2.70	3.90	
Migros Ouchy		4.03	7.10	3.20	1.02	2.22	2.95	2.67	2.57	2.24	2.83	
Alloboissons		77.03	80.32	73.17	71.69	71.67	76.60	73.66	73.76	77.42	73.11	

Étapes suivantes :

[Générer du code avec df_fleet](#)[New interactive sheet](#)

```
# Maps the long names to the short Distance Matrix names
customer_mapping = {
    "AlloBoissons SA": "Alloboissons",
    "Doki Doki - Mukmuk Sàrl": "Doki Doki",
    "Hospitality Partners Sàrl (Le Pointu)": "Pointu",
    "Les Docks": "Docks",
    "MKK SARL (Simplon)": "Simplon",
    "La Rincette Sàrl": "La rincette",
    "Jetée de la compagnie": "Jetée de la compagnie",
    "Migros Partenaire Ouchy Sàrl": "Migros Ouchy"
}

df_vrp_bulk["Customer"] = df_vrp_bulk["Customer"].str.strip().replace(customer_mapping)
df_vrp_standard["Customer"] = df_vrp_standard["Customer"].str.strip().replace(customer_mapping)

display(df_vrp_standard)

depot_name = "Entrepôt Crissier"
customer_names = df_vrp_standard["Customer"].tolist()
vrp_nodes = [depot_name] + customer_names
display(vrp_nodes)
df_vrp_matrix = df_distances.loc[vrp_nodes, vrp_nodes]
distance_matrix_meters = (df_vrp_matrix.values * 1000).astype(int)
```



```
display(df_vrp_matrix)
```

	Customer	Operational_Weeks	Weekly_Weight_kg	Weekly_Pallets	
0	Alloboissons	52.0	1394.59	1.77	
1	Doki Doki	52.0	128.25	0.16	
2	Great Escape	52.0	215.51	0.26	
3	Pointu	52.0	80.42	0.10	
4	Jetée de la compagnie	52.0	10.15	0.01	
5	La Latina	26.0	80.50	0.10	
6	La rincette	52.0	42.05	0.05	
7	Docks	52.0	291.93	0.36	
8	Loxton	52.0	49.77	0.06	
9	Simplon	12.0	280.17	0.34	
10	Migros Ouchy	52.0	78.44	0.10	
11	The Lacustre	52.0	169.50	0.21	

['Entrepôt Crissier',
'Alloboissons',
'Doki Doki',
'Great Escape',
'Pointu',
'Jetée de la compagnie',
'La Latina',
'La rincette',
'Docks',
'Loxton',
'Simplon',
'Migros Ouchy',
'The Lacustre']

	Entrepôt Crissier	Alloboissons	Doki Doki	Great Escape	Pointu	Jetée de la compagnie	La Latina	La rincette	Docks	Loxto
Origine \ Destination										
Entrepôt Crissier	0.00	80.71	7.00	7.17	6.19	8.6	6.55	7.6	5.77	6.6
Alloboissons	80.32	0.00	73.11	73.17	76.60	73.0	73.76	86.0	77.42	73.6
Doki Doki	8.96	73.92	0.00	0.69	2.08	3.2	0.84	3.9	2.44	0.8
Great Escape	9.22	74.34	0.69	0.00	1.48	3.8	0.96	3.6	2.70	0.8
Pointu	8.77	74.78	1.52	0.83	0.00	3.7	1.85	3.5	2.25	1.7
Jetée de la compagnie	8.60	73.00	3.20	3.80	3.70	0.0	3.10	2.1	2.70	3.2
La Latina	8.25	74.46	1.67	1.57	1.65	3.1	0.00	2.9	1.73	0.0
La rincette	7.60	86.00	3.90	3.60	3.50	2.1	2.90	0.0	2.70	3.0
Docks	7.02	75.83	2.44	2.61	1.85	2.7	1.74	2.7	0.00	1.8
Loxton	8.35	74.36	1.58	1.48	1.74	3.2	0.09	3.0	1.82	0.0
Simplon	8.31	72.57	1.50	1.87	2.46	1.6	2.08	2.7	2.43	1.5
Migros Ouchy	7.10	73.60	2.83	3.20	2.95	1.1	2.57	2.4	2.24	2.6
The Lacustre	7.91	72.58	2.34	2.71	3.30	1.1	2.92	2.4	3.28	2.3

Étapes

suivantes :

Générer du code avec df_vrp_standard

New interactive sheet

Générer du code avec df_vrp_matrix

✓ Solving the VRP: Heterogeneous Fleet Optimization

Scenario 1

```

str: url_distances
def create_data_model():
    """Stores the view for the routing problem."""
    data = {}
    data["node_names"] = vrp_nodes
    data["depot"] = 0
    data["distance_matrix"] = distance_matrix_meters

    # Demands in kg for each location
    data["demands"] = [0] + df_vrp_standard["Weekly_Weight_kg"].astype(int).tolist()

    # Vehicle capacities in kg
    data["vehicle_capacities"] = [700, 1200, 8300]
    data["num_vehicles"] = 3
    data["vehicle_names"] = ["VW Caddy", "Ford Traffic", "Truck"]
    data["vehicle_costs_cents"] = [60, 80, 150]

    return data

def print_solution(data, manager, routing, solution):
    """Prints the final solution to the console."""
    objective_value_chf = solution.ObjectiveValue() / 100.0
    print(f"Total Estimated Cost: {objective_value_chf:.2f} CHF\n")

    total_distance_meters = 0
    total_load = 0

    for vehicle_id in range(data["num_vehicles"]):
        index = routing.Start(vehicle_id)

        vehicle_name = data["vehicle_names"][vehicle_id]
        capacity = data["vehicle_capacities"][vehicle_id]

        plan_output = f"Route for {vehicle_name} (Capacity: {capacity} kg):\n"

        route_distance_meters = 0
        route_load = 0

        while not routing.IsEnd(index):
            node_index = manager.IndexToNode(index)
            node_name = data["node_names"][node_index]
            route_load += data["demands"][node_index]
            plan_output += f" {node_name} [{route_load} kg] ->"

            previous_index = index
            index = solution.Value(routing.NextVar(index))

            route_distance_meters += data["distance_matrix"][manager.IndexToNode(previous_index)][manager.IndexToNode(index)]

        node_index = manager.IndexToNode(index)
        node_name = data["node_names"][node_index]
        plan_output += f" {node_name} [{route_load} kg]"

        distance_km = route_distance_meters / 1000.0

        # Cost calculation
        route_cost_chf = (route_distance_meters * data["vehicle_costs_cents"][vehicle_id]) / 100000.0

        plan_output += f"\n Distance of the route: {distance_km:.2f} km\n"

```

```

plan_output += f" Cost of the route: {route_cost_chf:.2f} CHF\n"
plan_output += f" Total load of the route: {route_load} kg\n"
print(plan_output)

total_distance_meters += route_distance_meters
total_load += route_load

final_distance_km = total_distance_meters / 1000.0
print(f"Total distance of all routes: {final_distance_km:.2f} km")
print(f"Total load delivered: {total_load} kg")

def main():
    str: url_distances
    """Main execution function to solve the CVRP."""
    data = create_data_model(
        'https://raw.githubusercontent.com/amathie...'

    manager = pywrapcp.RoutingIndexManager(
        len(data["distance_matrix"]), data["num_vehicles"], data["depot"]
    )

    routing = pywrapcp.RoutingModel(manager)

    transit_callback_indices = []
    for v in range(data["num_vehicles"]):
        def cost_callback(from_index, to_index, vehicle_id=v):
            from_node = manager.IndexToNode(from_index)
            to_node = manager.IndexToNode(to_index)
            dist_m = data["distance_matrix"][from_node][to_node]
            return int((dist_m * data["vehicle_costs_cents"][vehicle_id]) / 1000)

        transit_callback_index = routing.RegisterTransitCallback(cost_callback)
        routing.SetArcCostEvaluatorOfVehicle(transit_callback_index, v)
        transit_callback_indices.append(transit_callback_index)

    # Capacity constraint
    def demand_callback(from_index):
        """Returns the demand of the node."""
        from_node = manager.IndexToNode(from_index)
        return data["demands"][from_node]

    demand_callback_index = routing.RegisterUnaryTransitCallback(demand_callback)

    routing.AddDimensionWithVehicleCapacity(
        demand_callback_index,
        0, # null capacity slack
        data["vehicle_capacities"], # array of vehicle capacities
        True, # start cumul to zero
        "Capacity"
    )

    search_parameters = pywrapcp.DefaultRoutingSearchParameters()
    search_parameters.first_solution_strategy = (
        routing_enums_pb2.FirstSolutionStrategy.PATH_CHEAPEST_ARC
    )
    search_parameters.local_search_metaheuristic = (
        routing_enums_pb2.LocalSearchMetaheuristic.GUIDED_LOCAL_SEARCH
    )
    search_parameters.time_limit.FromSeconds(10)

    # Solve
    print("Solving the VRP model (scenario 1). Please wait...\n")
    solution = routing.SolveWithParameters(search_parameters)

    # Print
    if solution:
        print_solution(data, manager, routing, solution)
    else:
        print("No solution found! Check your constraints and capacities.")

```

```
if __name__ == "__main__":
    main()
```

Solving the VRP model (scenario 1). Please wait...

Total Estimated Cost: 252.45 CHF

Route for VW Caddy (Capacity: 700 kg):

Entrepôt Crissier [0 kg] -> Entrepôt Crissier [0 kg]

Distance of the route: 0.00 km

Cost of the route: 0.00 CHF

Total load of the route: 0 kg

Route for Ford Traffic (Capacity: 1200 kg):

Entrepôt Crissier [0 kg] -> Entrepôt Crissier [0 kg]

Distance of the route: 0.00 km

Cost of the route: 0.00 CHF

Total load of the route: 0 kg

Route for Truck (Capacity: 8300 kg):

Entrepôt Crissier [0 kg] -> Docks [291 kg] -> Pointu [371 kg] -> Great Escape [586 kg] -> Doki Doki

Distance of the route: 168.33 km

Cost of the route: 252.50 CHF

Total load of the route: 2816 kg

Total distance of all routes: 168.33 km

Total load delivered: 2816 kg

✓ Scenario 2

```
def create_data_model():
    """Stores the data for the routing problem."""
    data = {}
    data["node_names"] = vrp_nodes
    data["depot"] = 0
    data["distance_matrix"] = distance_matrix_meters

    # Demands in kg for each location
    data["demands"] = [0] + df_vrp_standard["Weekly_Weight_kg"].astype(int).tolist()

    # Vehicle capacities in kg
    data["vehicle_capacities"] = [700, 1200, 8300]
    data["num_vehicles"] = 3
    data["vehicle_names"] = ["VW Caddy", "Ford Traffic", "Truck"]
    data["vehicle_costs_cents"] = [60, 80, 150]

    return data

def print_solution(data, manager, routing, solution):
    """Prints the final solution to the console."""
    objective_value_chf = solution.ObjectiveValue() / 100.0
    print(f"Total Estimated Cost: {objective_value_chf:.2f} CHF\n")

    total_distance_meters = 0
    total_load = 0

    for vehicle_id in range(data["num_vehicles"]):
        index = routing.Start(vehicle_id)

        vehicle_name = data["vehicle_names"][vehicle_id]
        capacity = data["vehicle_capacities"][vehicle_id]

        plan_output = f"Route for {vehicle_name} (Capacity: {capacity} kg):\n"

        route_distance_meters = 0
        route_load = 0

        while not routing.IsEnd(index):
```

```

        node_index = manager.IndexToNode(index)
        node_name = data["node_names"][node_index]
        route_load += data["demands"][node_index]
        plan_output += f" {node_name} [{route_load} kg] ->"

    previous_index = index
    index = solution.Value(routing.NextVar(index))

    route_distance_meters += data["distance_matrix"][manager.IndexToNode(previous_index)][manager.IndexToNode(index)]

    node_index = manager.IndexToNode(index)
    node_name = data["node_names"][node_index]
    plan_output += f" {node_name} [{route_load} kg]"

    distance_km = route_distance_meters / 1000.0

    # Cost calculation
    route_cost_chf = (route_distance_meters * data["vehicle_costs_cents"][vehicle_id]) / 100000.0

    plan_output += f"\n Distance of the route: {distance_km:.2f} km\n"
    plan_output += f" Cost of the route: {route_cost_chf:.2f} CHF\n"
    plan_output += f" Total load of the route: {route_load} kg\n"
    print(plan_output)

    total_distance_meters += route_distance_meters
    total_load += route_load

final_distance_km = total_distance_meters / 1000.0
print(f"Total distance of all routes: {final_distance_km:.2f} km")
print(f"Total load delivered: {total_load} kg")

def main():
    """Main execution function to solve the CVRP."""
    data = create_data_model()

    manager = pywrapcp.RoutingIndexManager(
        len(data["distance_matrix"]), data["num_vehicles"], data["depot"]
    )

    routing = pywrapcp.RoutingModel(manager)

    transit_callback_indices = []
    for v in range(data["num_vehicles"]):
        def cost_callback(from_index, to_index, vehicle_id=v):
            from_node = manager.IndexToNode(from_index)
            to_node = manager.IndexToNode(to_index)
            dist_m = data["distance_matrix"][from_node][to_node]
            return int((dist_m * data["vehicle_costs_cents"][vehicle_id]) / 1000)

        transit_callback_index = routing.RegisterTransitCallback(cost_callback)
        routing.SetArcCostEvaluatorOfVehicle(transit_callback_index, v)
        transit_callback_indices.append(transit_callback_index)

    # Capacity constraint
    def demand_callback(from_index):
        """Returns the demand of the node."""
        from_node = manager.IndexToNode(from_index)
        return data["demands"][from_node]

    demand_callback_index = routing.RegisterUnaryTransitCallback(demand_callback)

    routing.AddDimensionWithVehicleCapacity(
        demand_callback_index,
        0, # null capacity slack
        data["vehicle_capacities"], # array of vehicle capacities
        True, # start cumul to zero
        "Capacity"
    )

```

```

# CONSTRAINT: EXCLUSION OF TRUCK FROM CITY CENTER
# Nodes 2 through 12 are urban bars/restaurants.
# We add a constraint directly to the solver stating that the vehicle
# assigned to these nodes CANNOT be Vehicle 2 (the Truck).
for i in range(2, 13):
    index = manager.NodeToIndex(i)
    routing.solver().Add(routing.VehicleVar(index) != 2)

search_parameters = pywrapcp.DefaultRoutingSearchParameters()
search_parameters.first_solution_strategy = routing_enums_pb2.FirstSolutionStrategy.PATH_CHEAPEST
search_parameters.local_search_metaheuristic = routing_enums_pb2.LocalSearchMetaheuristic.GUIDED
search_parameters.time_limit.FromSeconds(10)

print("Solving the VRP model(scenario 2). Please wait...\n")
solution = routing.SolveWithParameters(search_parameters)

if solution:
    print_solution(data, manager, routing, solution)
else:
    print("No solution found!")

if __name__ == "__main__":
    main()

```

Solving the VRP model(scenario 2). Please wait...

Total Estimated Cost: 267.78 CHF

Route for VW Caddy (Capacity: 700 kg):

Entrepôt Crissier [0 kg] -> Simplon [280 kg] -> The Lacustre [449 kg] -> Migros Ouchy [527 kg] -> Jet
 Distance of the route: 20.54 km
 Cost of the route: 12.32 CHF
 Total load of the route: 579 kg

Route for Ford Traffic (Capacity: 1200 kg):

Entrepôt Crissier [0 kg] -> Pointu [80 kg] -> Great Escape [295 kg] -> Doki Doki [423 kg] -> Loxton
 Distance of the route: 17.43 km
 Cost of the route: 13.94 CHF
 Total load of the route: 843 kg

Route for Truck (Capacity: 8300 kg):

Entrepôt Crissier [0 kg] -> Alloboissons [1394 kg] -> Entrepôt Crissier [1394 kg]
 Distance of the route: 161.03 km
 Cost of the route: 241.54 CHF
 Total load of the route: 1394 kg

Total distance of all routes: 199.00 km

Total load delivered: 2816 kg

✓ Scenario 3

```

def create_flexible_data_model():
    """Stores data featuring multiple tours and relaxed truck constraints."""
    data = {}
    data["node_names"] = vrp_nodes
    data["depot"] = 0
    data["distance_matrix"] = distance_matrix_meters
    data["demands"] = [0] + df_vrp_standard["Weekly_Weight_kg"].astype(int).tolist()

    # Adding "Tour 2" virtual vehicles
    data["vehicle_capacities"] = [700, 700, 1200, 1200, 8300]
    data["num_vehicles"] = 5
    data["vehicle_names"] = [
        "VW Caddy (Tour 1)",
        "VW Caddy (Tour 2)",
        "Ford Transit (Tour 1)",
        "Ford Transit (Tour 2)",
        "Truck"
    ]

```

```

    ...]
    data["vehicle_costs_cents"] = [60, 60, 80, 80, 150]

    ...return data

def print_flexible_solution(data, manager, routing, solution):
    """Prints the final solution to the console."""
    objective_value_chf = solution.ObjectiveValue() / 100.0
    print(f"OPTIMIZED ROUTING")
    print(f"Total Estimated Weekly Cost: {objective_value_chf:.2f} CHF\n")

    str: url_distances
    total_distance_meters = 0
    total_load = 0

    for vehicle_id in range(data["num_vehicles"]):
        index = routing.Start(vehicle_id)

        vehicle_name = data["vehicle_names"][vehicle_id]
        capacity = data["vehicle_capacities"][vehicle_id]
        plan_output = f"Route for {vehicle_name} (Capacity: {capacity} kg):\n"

        route_distance_meters = 0
        route_load = 0

        while not routing.IsEnd(index):
            node_index = manager.IndexToNode(index)
            node_name = data["node_names"][node_index]
            route_load += data["demands"][node_index]
            plan_output += f"{node_name} [{route_load} kg] -> "

            previous_index = index
            index = solution.Value(routing.NextVar(index))
            route_distance_meters += data["distance_matrix"][manager.IndexToNode(previous_index)][manager.IndexToNode(index)]

            node_index = manager.IndexToNode(index)
            node_name = data["node_names"][node_index]
            plan_output += f"{node_name} [{route_load} kg]"

        distance_km = route_distance_meters / 1000.0
        route_cost_chf = (route_distance_meters * data["vehicle_costs_cents"][vehicle_id]) / 100000.0

        plan_output += f"\n -> Distance: {distance_km:.2f} km | Cost: {route_cost_chf:.2f} CHF | Load: {route_load} kg\n"
        print(plan_output)

        total_distance_meters += route_distance_meters
        total_load += route_load

    final_distance_km = total_distance_meters / 1000.0
    print("-" * 40)
    print(f"TOTAL DISTANCE: {final_distance_km:.2f} km")
    print(f"TOTAL LOAD: {total_load} kg")
    print("-" * 40)

def main_flexible():
    data = create_flexible_data_model()
    manager = pywrapcp.RoutingIndexManager(len(data["distance_matrix"]), data["num_vehicles"], data["demands"])
    routing = pywrapcp.RoutingModel(manager)

    transit_callback_indices = []
    for v in range(data["num_vehicles"]):
        def cost_callback(from_index, to_index, vehicle_id=v):
            from_node = manager.IndexToNode(from_index)
            to_node = manager.IndexToNode(to_index)
            dist_m = data["distance_matrix"][from_node][to_node]
            return int((dist_m * data["vehicle_costs_cents"][vehicle_id]) / 1000)

        transit_callback_index = routing.RegisterTransitCallback(cost_callback)
        routing.SetArcCostEvaluatorOfVehicle(transit_callback_index, v)
        transit_callback_indices.append(transit_callback_index)

```

```

transit_callback_indices.append(transit_callback_index)

def demand_callback(from_index):
    from_node = manager.IndexToNode(from_index)
    return data["demands"][from_node]

demand_callback_index = routing.RegisterUnaryTransitCallback(demand_callback)
routing.AddDimensionWithVehicleCapacity(
    demand_callback_index, 0, data["vehicle_capacities"], True, "Capacity"
)

# --- SMART URBAN EXCLUSION CONSTRAINT ---
# The Truck is allowed to visit the Depot, Alloboissons, Migros Ouchy, and The Lacustre.
allowed_truck_nodes = [
    "Entrepôt Crissier",
    "Alloboissons",
    "Migros Ouchy",
    "The Lacustre",
    "Jetée de la compagnie",
    "La rincette"
]

for i in range(len(data["node_names"])):
    node_name = data["node_names"][i]
    if node_name not in allowed_truck_nodes:
        index = manager.NodeToIndex(i)
        routing.VehicleVar(index).RemoveValue(4)

search_parameters = pywrapcp.DefaultRoutingSearchParameters()
search_parameters.first_solution_strategy = routing_enums_pb2.FirstSolutionStrategy.PATH_CHEAPEST_
search_parameters.local_search_metaheuristic = routing_enums_pb2.LocalSearchMetaheuristic.GUIDED_L
search_parameters.time_limit.FromSeconds(10)

print("Solving the VRP model (scenario 3)). Please wait...\n")
solution = routing.SolveWithParameters(search_parameters)

if solution:
    print_flexible_solution(data, manager, routing, solution)
else:
    print("No solution found! The constraints are still too tight.")

if __name__ == "__main__":
    main_flexible()

```

Solving the VRP model (scenario 3)). Please wait...

OPTIMIZED ROUTING

Total Estimated Weekly Cost: 259.36 CHF

Route for VW Caddy (Tour 1) (Capacity: 700 kg):

Entrepôt Crissier [0 kg] -> Entrepôt Crissier [0 kg]
 > Distance: 0.00 km | Cost: 0.00 CHF | Load: 0 kg

Route for VW Caddy (Tour 2) (Capacity: 700 kg):

Entrepôt Crissier [0 kg] -> Entrepôt Crissier [0 kg]
 > Distance: 0.00 km | Cost: 0.00 CHF | Load: 0 kg

Route for Ford Transit (Tour 1) (Capacity: 1200 kg):

Entrepôt Crissier [0 kg] -> Pointu [80 kg] -> Great Escape [295 kg] -> Doki Doki [423 kg] -> Sir
 > Distance: 19.48 km | Cost: 15.58 CHF | Load: 1123 kg

Route for Ford Transit (Tour 2) (Capacity: 1200 kg):

Entrepôt Crissier [0 kg] -> Entrepôt Crissier [0 kg]
 > Distance: 0.00 km | Cost: 0.00 CHF | Load: 0 kg

Route for Truck (Capacity: 8300 kg):

Entrepôt Crissier [0 kg] -> La rincette [42 kg] -> Jetée de la compagnie [52 kg] -> Alloboissons
 > Distance: 162.54 km | Cost: 243.81 CHF | Load: 1693 kg

 TOTAL DISTANCE: 182.02 km

TOTAL LOAD: 2816 kg

Maps

as-the-crow-flies

```
def create_route_map():
    """Generates an interactive map of the final optimized routes (Scenario 3)."""
    str: url_distances

    # 1. GPS Coordinates for all 12 nodes (Latitude, Longitude)
    # These are accurate approximations based on the provided addresses.
    locations = { 'https://raw.githubusercontent.com/amathie...'
        0: {"name": "Brasserie Renens", "coords": [46.5324, 6.5904]},
        1: {"name": "Entrepot Crissier (Depot)", "coords": [46.5511, 6.5709]},
        2: {"name": "Great Escape", "coords": [46.5229, 6.6332]},
        3: {"name": "The Lacustre", "coords": [46.5050, 6.6273]},
        4: {"name": "Simplon", "coords": [46.5158, 6.6300]},
        5: {"name": "Pointu", "coords": [46.5234, 6.6308]},
        6: {"name": "Loxton", "coords": [46.5212, 6.6333]},
        7: {"name": "La Latina", "coords": [46.5209, 6.6321]},
        8: {"name": "Docks", "coords": [46.5225, 6.6192]},
        9: {"name": "Doki Doki", "coords": [46.5201, 6.6401]},
        10: {"name": "Jetée de la compagnie", "coords": [46.5068, 6.6192]},
        11: {"name": "La Rincette", "coords": [46.5122, 6.6021]},
        12: {"name": "Migros Ouchy", "coords": [46.5077, 6.6247]},
        13: {"name": "Alloboissons", "coords": [46.8182, 7.1330]}
    }

    # 2. Hardcoded optimal routes extracted from Scenario 3 solver output
    route_caddy = [1, 6, 7, 8, 1]
    route_ford = [1, 5, 2, 9, 4, 10, 11, 1]
    route_truck = [1, 13, 3, 12, 1]

    # 3. Initialize map centered between Lausanne and Givisiez
    m = folium.Map(location=[46.65, 6.8], zoom_start=10, tiles="CartoDB positron")

    # 4. Add markers for all nodes
    for index, data in locations.items():
        if index == 1:
            # Depot in Black
            icon_color = "black"
            icon_type = "home"
        elif index == 0:
            # Production in Purple
            icon_color = "purple"
            icon_type = "industry"
        elif index == 11:
            # Bulk Distributor in Red
            icon_color = "red"
            icon_type = "shopping-cart"
        else:
            # Urban clients in Blue
            icon_color = "blue"
            icon_type = "glass"

        folium.Marker(
            location=data["coords"],
            popup=f"[{index}] {data['name']}",
            tooltip=data["name"],
            icon=folium.Icon(color=icon_color, icon=icon_type, prefix="fa")
        ).add_to(m)

    # 5. Draw the routes
    # Extract coordinate lists for each route
    caddy_coords = [locations[node]["coords"] for node in route_caddy]
    traffic_coords = [locations[node]["coords"] for node in route_ford]
```

```

truck_coords = [locations[node]["coords"] for node in route_truck]

# Add lines to the map
folium.PolyLine(
    caddy_coords,
    color="green",
    weight=4,
    opacity=0.8,
    tooltip="VW Caddy Route (700 kg)"
).add_to(m)

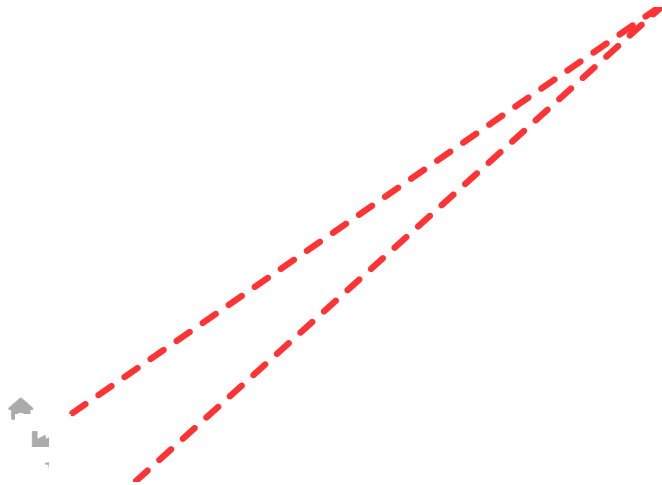
folium.PolyLine(
    traffic_coords,
    color="orange",
    weight=4,
    opacity=0.8,
    tooltip="Ford Traffic Route (1200 kg)"
).add_to(m)

folium.PolyLine(
    truck_coords,
    color="red",
    weight=4,
    opacity=0.8,
    dash_array="10",
    tooltip="Heavy Truck Route (8300 kg)"
).add_to(m)

return m

# Display the map in the notebook
my_map = create_route_map()
my_map

```



Leaflet | © OpenStreetMap contributors © CARTO

✓ Real world road network

```

def get_osrm_route(coords_list):
    """
    Interroge l'API OSRM pour obtenir le vrai tracé routier entre plusieurs points.
    coords_list: liste de listes [latitude, longitude]
    """

```

```

"""
# L'API OSRM a besoin des coordonnees au format "longitude,latitude"
waypoints = ";\n".join([f"{lon},{lat}" for lat, lon in coords_list])

# URL de l'API publique OSRM (geometries=geojson pour eviter de devoir decoder)
url = f"http://router.project-osrm.org/route/v1/driving/{waypoints}?overview=full&geometries=geojson"

try:
    response = requests.get(url)
    if response.status_code == 200:
        data = response.json()
        # OSRM renvoie [longitude, latitude], Folium a besoin de [latitude, longitude]
        route_coords = data['routes'][0]['geometry']['coordinates']
        return [[lat, lon] for lon, lat in route_coords]
    else:
        print("Note: Impossible de joindre OSRM. Retour aux lignes droites.")
        return coords_list
except Exception as e:
    print(f"Erreur de connexion a OSRM: {e}. Retour aux lignes droites.")
    return coords_list

def create_real_route_map():
    """Generates an interactive map with actual driving routes."""

    # 1. GPS Coordinates
    locations = {
        0: {"name": "Brasserie Renens", "coords": [46.5324, 6.5904]},
        1: {"name": "Entrepot Crissier (Depot)", "coords": [46.5511, 6.5709]},
        2: {"name": "Great Escape", "coords": [46.5229, 6.6332]},
        3: {"name": "The Lacustre", "coords": [46.5050, 6.6273]},
        4: {"name": "Simplon", "coords": [46.5158, 6.6300]},
        5: {"name": "Pointu", "coords": [46.5234, 6.6308]},
        6: {"name": "Loxton", "coords": [46.5212, 6.6333]},
        7: {"name": "La Latina", "coords": [46.5209, 6.6321]},
        8: {"name": "Docks", "coords": [46.5225, 6.6192]},
        9: {"name": "Doki Doki", "coords": [46.5201, 6.6401]},
        10: {"name": "Jetée de la compagnie", "coords": [46.5068, 6.6192]},
        11: {"name": "La Rincette", "coords": [46.5122, 6.6021]},
        12: {"name": "Migros Ouchy", "coords": [46.5077, 6.6247]},
        13: {"name": "Alloboissons", "coords": [46.8182, 7.1330]}
    }

    route_caddy = [1, 6, 7, 8, 1]
    route_ford = [1, 5, 2, 9, 4, 10, 11, 1]
    route_truck = [1, 13, 3, 12, 1]

    # Initialize map
    m = folium.Map(location=[46.65, 6.8], zoom_start=10, tiles="CartoDB positron")

    # Add markers
    for index, data in locations.items():
        if index == 1:
            icon_color, icon_type = "black", "home"
        elif index == 0:
            icon_color, icon_type = "purple", "industry"
        elif index == 11:
            icon_color, icon_type = "red", "shopping-cart"
        else:
            icon_color, icon_type = "blue", "glass"

        folium.Marker(
            location=data["coords"],
            popup=f"[{index}] {data['name']}",
            tooltip=data["name"],
            icon=folium.Icon(color=icon_color, icon=icon_type, prefix="fa")
        ).add_to(m)

    # Get REAL driving routes from OSRM
    print("Calcul des itineraires routiers en cours...")

```

```

caddy_real_coords = get_osrm_route([locations[node]["coords"] for node in route_caddy])
traffic_real_coords = get_osrm_route([locations[node]["coords"] for node in route_ford])
truck_real_coords = get_osrm_route([locations[node]["coords"] for node in route_truck])

# Draw the real paths on the map
folium.PolyLine(
    caddy_real_coords,
    color="green",
    weight=4,
    opacity=0.8,
    tooltip="VW Caddy Route (700 kg)"
).add_to(m)

    View
folium.PolyLine(https://raw.githubusercontent.com/amathie5/20544258974c267d37f17c1409612784/sl\_project\_vf-ev.ipynb#scrollTo=32bSGrTX2kFY&print...,
    traffic_real_coords,
    color="orange",
    weight=4,
    opacity=0.8,
    tooltip="Ford Traffic Route (1200 kg)"
).add_to(m)

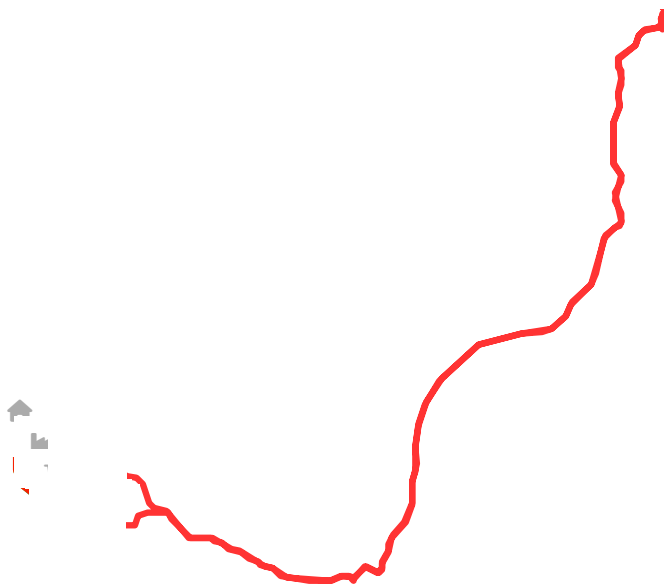
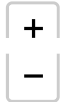
folium.PolyLine(
    truck_real_coords,
    color="red",
    weight=4,
    opacity=0.8,
    tooltip="Heavy Truck Route (8300 kg)"
).add_to(m)

print("Carte generee avec succes !")
return m

# Display the map in the notebook
my_real_map = create_real_route_map()
my_real_map

```

Calcul des itineraires routiers en cours...
Carte generee avec succes !



Leaflet | © OpenStreetMap contributors © CARTO

✓ Scenario 4

```

def create_flexible_data_model():
    """Stores data featuring multiple tours and relaxed truck constraints."""
    data = {}
    data["node_names"] = vrp_nodes
    data["depot"] = 0
    data["distance_matrix"] = distance_matrix_meters
    data["demands"] = [0] + df_vrp_standard["Weekly_Weight_kg"].astype(int).tolist()

    # Adding "Tour 2" virtual vehicles
    data["vehicle_capacities"] = [700, 700, 1200, 1200, 8300]
    data["num_vehicles"] = 5
    data["vehicle_names"] = [
        "VW Caddy (Tour 1)",
        "VW Caddy (Tour 2)",
        "Ford Transit (Tour 1)",
        "Ford Transit (Tour 2)",
        "Truck"
    ]
    data["vehicle_costs_cents"] = [60, 60, 80, 80, 150]

    return data

def print_flexible_solution(data, manager, routing, solution):
    """Prints the final solution to the console."""
    objective_value_chf = solution.ObjectiveValue() / 100.0
    print(f"OPTIMIZED ROUTING")
    print(f"Total Estimated Weekly Cost: {objective_value_chf:.2f} CHF\n")

    total_distance_meters = 0
    total_load = 0

    for vehicle_id in range(data["num_vehicles"]):
        index = routing.Start(vehicle_id)

        vehicle_name = data["vehicle_names"][vehicle_id]
        capacity = data["vehicle_capacities"][vehicle_id]
        plan_output = f"Route for {vehicle_name} (Capacity: {capacity} kg):\n"

        route_distance_meters = 0
        route_load = 0

        while not routing.IsEnd(index):
            node_index = manager.IndexToNode(index)
            node_name = data["node_names"][node_index]
            route_load += data["demands"][node_index]
            plan_output += f" {node_name} [{route_load} kg] ->"

            previous_index = index
            index = solution.Value(routing.NextVar(index))
            route_distance_meters += data["distance_matrix"][manager.IndexToNode(previous_index)][manager.IndexToNode(index)]

        node_index = manager.IndexToNode(index)
        node_name = data["node_names"][node_index]
        plan_output += f" {node_name} [{route_load} kg]"

        distance_km = route_distance_meters / 1000.0
        route_cost_chf = (route_distance_meters * data["vehicle_costs_cents"][vehicle_id]) / 100000.0

        plan_output += f"\n > Distance: {distance_km:.2f} km | Cost: {route_cost_chf:.2f} CHF | Load: {route_load} kg"
        print(plan_output)

        total_distance_meters += route_distance_meters
        total_load += route_load

    final_distance_km = total_distance_meters / 1000.0
    print("-" * 40)
    print(f"TOTAL DISTANCE: {final_distance_km:.2f} km")
    print(f"TOTAL LOAD: {total_load} kg")
    print("-" * 40)

```

```

def main_forced_lakeside():
    data = create_flexible_data_model()
    manager = pywrapcp.RoutingIndexManager(len(data["distance_matrix"]), data["num_vehicles"], data[
    routing = pywrapcp.RoutingModel(manager)

    # Dynamic Cost Callbacks
    transit_callback_indices = []
    for v in range(data["num_vehicles"]):
        def cost_callback(from_index, to_index, vehicle_id=v):
            from_node = manager.IndexToNode(from_index)
            to_node = manager.IndexToNode(to_index)
            dist_m = data["distance_matrix"][from_node][to_node]
            return int((dist_m * data["vehicle_costs_cents"][vehicle_id]) / 1000)

        transit_callback_index = routing.RegisterTransitCallback(cost_callback)
        routing.SetArcCostEvaluatorOfVehicle(transit_callback_index, v)
        transit_callback_indices.append(transit_callback_index)

    # Capacity constraint
    def demand_callback(from_index):
        from_node = manager.IndexToNode(from_index)
        return data["demands"][from_node]

    demand_callback_index = routing.RegisterUnaryTransitCallback(demand_callback)
    routing.AddDimensionWithVehicleCapacity(
        demand_callback_index, 0, data["vehicle_capacities"], True, "Capacity"
    )

    # We keep the urban exclusion zone (Truck cannot go to dense city center)
    allowed_truck_nodes = [
        "Entrepôt Crissier",
        "Alloboissons",
        "Migros Ouchy",
        "The Lacustre",
        "Jetée de la compagnie",
        "La rincette"
    ]

    for i in range(len(data["node_names"])):
        node_name = data["node_names"][i]
        if node_name not in allowed_truck_nodes:
            index = manager.NodeToIndex(i)
            routing.VehicleVar(index).RemoveValue(4) # 4 is the Truck

    # WE FORCE THE TRUCK TO TAKE THE LAKESIDE NODES
    nodes_to_force_on_truck = [
        "Migros Ouchy",
        "The Lacustre",
        "Jetée de la compagnie",
        "La rincette"
    ]

    for node_name in nodes_to_force_on_truck:
        if node_name in data["node_names"]:
            node_idx = data["node_names"].index(node_name)
            manager_idx = manager.NodeToIndex(node_idx)
            routing.VehicleVar(manager_idx).SetValue(4)

    search_parameters = pywrapcp.DefaultRoutingSearchParameters()
    search_parameters.first_solution_strategy = routing_enums_pb2.FirstSolutionStrategy.PATH_CHEAPEST
    search_parameters.local_search_metaheuristic = routing_enums_pb2.LocalSearchMetaheuristic.GUIDED
    search_parameters.time_limit.FromSeconds(10)

    print("Solving the VRP model (scenario 4). Please wait...\n")
    solution = routing.SolveWithParameters(search_parameters)

    if solution:

```

```

print_flexible_solution(data, manager, routing, solution)
else:
    print("No solution found! Forcing these nodes might violate the Truck's capacity.")

main_forced_lakeside()

Solving the VRP model (scenario 4). Please wait...

OPTIMIZED ROUTING
Total Estimated Weekly Cost: 259.36 CHF

Route for VW Caddy (Tour 1) (Capacity: 700 kg):
  Entrepôt Crissier [0 kg] -> Entrepôt Crissier [0 kg]
  > Distance: 0.00 km | Cost: 0.00 CHF | Load: 0 kg

Route for VW Caddy (Tour 2) (Capacity: 700 kg):
  Entrepôt Crissier [0 kg] -> Entrepôt Crissier [0 kg]
  > Distance: 0.00 km | Cost: 0.00 CHF | Load: 0 kg

Route for Ford Transit (Tour 1) (Capacity: 1200 kg):
  Entrepôt Crissier [0 kg] -> Pointu [80 kg] -> Great Escape [295 kg] -> Doki Doki [423 kg] -> Sir
  > Distance: 19.48 km | Cost: 15.58 CHF | Load: 1123 kg

Route for Ford Transit (Tour 2) (Capacity: 1200 kg):
  Entrepôt Crissier [0 kg] -> Entrepôt Crissier [0 kg]
  > Distance: 0.00 km | Cost: 0.00 CHF | Load: 0 kg

Route for Truck (Capacity: 8300 kg):
  Entrepôt Crissier [0 kg] -> La rincette [42 kg] -> Jetée de la compagnie [52 kg] -> Alloboissons
  > Distance: 162.54 km | Cost: 243.81 CHF | Load: 1693 kg

-----
TOTAL DISTANCE: 182.02 km
TOTAL LOAD: 2816 kg
-----

```

✓ Scenario 5

```

def main_forced_sequence():
    data = create_flexible_data_model()
    manager = pywrapcp.RoutingIndexManager(len(data["distance_matrix"]), data["num_vehicles"], data[
    routing = pywrapcp.RoutingModel(manager)

    transit_callback_indices = []
    for v in range(data["num_vehicles"]):
        def cost_callback(from_index, to_index, vehicle_id=v):
            from_node = manager.IndexToNode(from_index)
            to_node = manager.IndexToNode(to_index)
            dist_m = data["distance_matrix"][from_node][to_node]
            return int((dist_m * data["vehicle_costs_cents"][vehicle_id]) / 1000)

        transit_callback_index = routing.RegisterTransitCallback(cost_callback)
        routing.SetArcCostEvaluatorOfVehicle(transit_callback_index, v)
        transit_callback_indices.append(transit_callback_index)

    # Capacity constraint
    def demand_callback(from_index):
        from_node = manager.IndexToNode(from_index)
        return data["demands"][from_node]

    demand_callback_index = routing.RegisterUnaryTransitCallback(demand_callback)
    routing.AddDimensionWithVehicleCapacity(
        demand_callback_index, 0, data["vehicle_capacities"], True, "Capacity"
    )

    # Keep the truck out of the city center
    allowed_truck_nodes = [
        "Entrepôt Crissier",
        "Alloboissons",

```

```

    "Migros Ouchy",
    "The Lacustre",
    "Jetée de la compagnie",
    "La rincette"
]

for i in range(len(data["node_names"])):
    node_name = data["node_names"][i]
    if node_name not in allowed_truck_nodes:
        index = manager.NodeToIndex(i)
        routing.VehicleVar(index).RemoveValue(4) # 4 is the Truck
        str: url_distances

# FORCED ALLOCATION
nodes_to_force_on_truck = [
    "Migros Ouchy",
    "The Lacustre",
    "Jetée de la compagnie",
    "La rincette"
]

for node_name in nodes_to_force_on_truck:
    if node_name in data["node_names"]:
        node_idx = data["node_names"].index(node_name)
        manager_idx = manager.NodeToIndex(node_idx)
        routing.VehicleVar(manager_idx).SetValue(4)

# FORCED SEQUENCE: TRUCK MUST GO TO GIVISIEZ (ALLOBOISSONS) FIRST
truck_id = 4
if "Alloboissons" in data["node_names"]:
    alloboissons_node_idx = data["node_names"].index("Alloboissons")
    manager_alloboissons_idx = manager.NodeToIndex(alloboissons_node_idx)

    routing.solver().Add(routing.NextVar(routing.Start(truck_id)) == manager_alloboissons_idx)

search_parameters = pywrapcp.DefaultRoutingSearchParameters()
search_parameters.first_solution_strategy = routing_enums_pb2.FirstSolutionStrategy.PARALLEL_CHE
search_parameters.local_search_metaheuristic = routing_enums_pb2.LocalSearchMetaheuristic.GUIDED
search_parameters.time_limit.FromSeconds(10)

print("Solving the VRP model (scenario 5). Please wait...\n")
solution = routing.SolveWithParameters(search_parameters)

if solution:
    print_flexible_solution(data, manager, routing, solution)
else:
    print("No solution found! The sequence constraint might violate logic or capacity.")

main_forced_sequence()

```

Solving the VRP model (scenario 5). Please wait...

OPTIMIZED ROUTING

Total Estimated Weekly Cost: 261.92 CHF

Route for VW Caddy (Tour 1) (Capacity: 700 kg):

Entrepôt Crissier [0 kg] -> Entrepôt Crissier [0 kg]
 > Distance: 0.00 km | Cost: 0.00 CHF | Load: 0 kg

Route for VW Caddy (Tour 2) (Capacity: 700 kg):

Entrepôt Crissier [0 kg] -> Entrepôt Crissier [0 kg]
 > Distance: 0.00 km | Cost: 0.00 CHF | Load: 0 kg

Route for Ford Transit (Tour 1) (Capacity: 1200 kg):

Entrepôt Crissier [0 kg] -> Pointu [80 kg] -> Great Escape [295 kg] -> Doki Doki [423 kg] -> Sir
 > Distance: 19.48 km | Cost: 15.58 CHF | Load: 1123 kg

Route for Ford Transit (Tour 2) (Capacity: 1200 kg):

Entrepôt Crissier [0 kg] -> Entrepôt Crissier [0 kg]
 > Distance: 0.00 km | Cost: 0.00 CHF | Load: 0 kg

Route for Truck (Capacity: 8300 kg):


```
Entrepôt Crissier [0 kg] -> Alloboissons [1394 kg] -> The Lacustre [1563 kg] -> Migros Ouchy [164
> Distance: 164.25 km | Cost: 246.38 CHF | Load: 1693 kg
```

```
-----
TOTAL DISTANCE: 183.73 km
```

```
TOTAL LOAD: 2816 kg
-----
```

✦ Itinerary map

```
str: url_distances
```

```
def get_osrm_route(coords_list):
    """
    'https://raw.githubusercontent.com/amathi...'
    Interroge l'API OSRM pour obtenir le vrai tracé routier entre plusieurs points.
    coords_list: liste de listes [latitude, longitude]
    """
    # L'API OSRM a besoin des coordonnées au format "longitude,latitude"
    waypoints = " ".join([f"{lon},{lat}" for lat, lon in coords_list])

    # URL de l'API publique OSRM (geometries=geojson pour éviter de devoir decoder)
    url = f"http://router.project-osrm.org/route/v1/driving/{waypoints}?overview=full&geometries=geo"

    try:
        response = requests.get(url)
        if response.status_code == 200:
            data = response.json()
            # OSRM renvoie [longitude, latitude], Folium a besoin de [latitude, longitude]
            route_coords = data['routes'][0]['geometry']['coordinates']
            return [[lat, lon] for lon, lat in route_coords]
        else:
            print("Note: Impossible de joindre OSRM. Retour aux lignes droites.")
            return coords_list
    except Exception as e:
        print(f"Erreur de connexion a OSRM: {e}. Retour aux lignes droites.")
        return coords_list

def create_real_route_map():
    """Generates an interactive map with actual driving routes."""

    # 1. GPS Coordinates
    locations = {
        0: {"name": "Brasserie Renens", "coords": [46.5324, 6.5904]},
        1: {"name": "Entrepot Crissier (Depot)", "coords": [46.5511, 6.5709]},
        2: {"name": "Great Escape", "coords": [46.5229, 6.6332]},
        3: {"name": "The Lacustre", "coords": [46.5050, 6.6273]},
        4: {"name": "Simplon", "coords": [46.5158, 6.6300]},
        5: {"name": "Pointu", "coords": [46.5234, 6.6308]},
        6: {"name": "Loxton", "coords": [46.5212, 6.6333]},
        7: {"name": "La Latina", "coords": [46.5209, 6.6321]},
        8: {"name": "Docks", "coords": [46.5225, 6.6192]},
        9: {"name": "Doki Doki", "coords": [46.5201, 6.6401]},
        10: {"name": "Jetée de la compagnie", "coords": [46.5068, 6.6192]},
        11: {"name": "La Rincette", "coords": [46.5122, 6.6021]},
        12: {"name": "Migros Ouchy", "coords": [46.5077, 6.6247]},
        13: {"name": "Alloboissons", "coords": [46.8182, 7.1330]}
    }

    route_caddy = [1, 8, 1]
    route_ford = [1, 5, 2, 9, 4, 6, 7, 1]
    route_truck = [1, 13, 3, 12, 10, 11, 1]

    # Initialize map
    m = folium.Map(location=[46.65, 6.8], zoom_start=10, tiles="CartoDB positron")

    # Add markers
    for index, data in locations.items():
        if index == 1:
            icon_color, icon_type = "black", "home"
```

```

elif index == 0:
    icon_color, icon_type = "purple", "industry"
elif index == 11:
    icon_color, icon_type = "red", "shopping-cart"
else:
    icon_color, icon_type = "blue", "glass"

folium.Marker(
    location=data["coords"],
    popup=f"[{index}] {data['name']}",
    tooltip=data["name"],
    icon=folium.Icon(color=icon_color, icon=icon_type, prefix="fa")
).add_to(m)

# Get REAL driving routes from OSRM
print("Calcul des itineraires routiers en cours...")
caddy_real_coords = get_osrm_route([locations[node]["coords"] for node in route_caddy])
traffic_real_coords = get_osrm_route([locations[node]["coords"] for node in route_ford])
truck_real_coords = get_osrm_route([locations[node]["coords"] for node in route_truck])

# Draw the real paths on the map
folium.PolyLine(
    caddy_real_coords,
    color="green",
    weight=4,
    opacity=0.8,
    tooltip="VW Caddy Route (700 kg)"
).add_to(m)

folium.PolyLine(
    traffic_real_coords,
    color="orange",
    weight=4,
    opacity=0.8,
    tooltip="Ford Traffic Route (1200 kg)"
).add_to(m)

folium.PolyLine(
    truck_real_coords,
    color="red",
    weight=4,
    opacity=0.8,
    tooltip="Heavy Truck Route (8300 kg)"
).add_to(m)

print("Carte genereee avec succes !")
return m

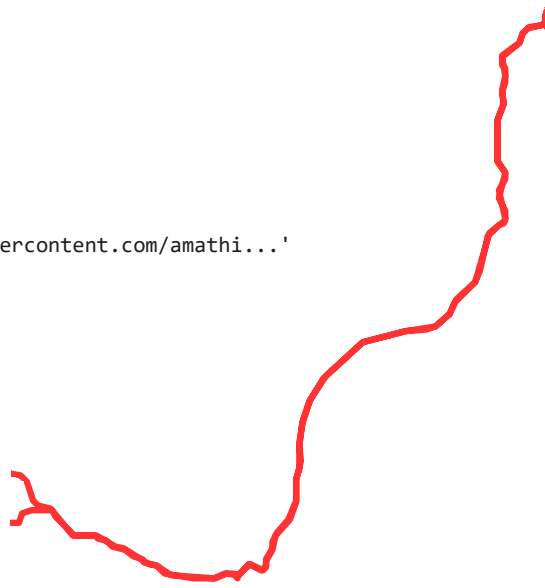
# Display the map in the notebook
my_real_map = create_real_route_map()
my_real_map

```

Calcul des itineraires routiers en cours...
Carte generee avec succes !



```
str: url_distances
View
'https://raw.githubusercontent.com/amathie5/20544258974c267d37f17c1409612784/sl_project_vf-ev.ipynb#scrollTo=32bSGrTX2kFY&print...
```



Leaflet | © OpenStreetMap contributors © CARTO

✓ Sensitivity Analysis

Demand Growth & Fleet Resilience

```
def evaluate_demand_scenario(multiplier):
    """Runs the Scenario 5 VRP (Forced Lakeside + Sequence) for a specific demand multiplier."""
    data = create_flexible_data_model()

    # Scale the demands by the multiplier (simulating sales growth)
    data["demands"] = [0] + (df_vrp_standard["Weekly_Weight_kg"] * multiplier).astype(int).tolist()

    manager = pywrapcp.RoutingIndexManager(len(data["distance_matrix"]), data["num_vehicles"], data[
    routing = pywrapcp.RoutingModel(manager)

    # 1. Costs
    transit_callback_indices = []
    for v in range(data["num_vehicles"]):
        def cost_callback(from_index, to_index, vehicle_id=v):
            from_node = manager.IndexToNode(from_index)
            to_node = manager.IndexToNode(to_index)
            dist_m = data["distance_matrix"][from_node][to_node]
            return int((dist_m * data["vehicle_costs_cents"][vehicle_id]) / 1000)

        idx = routing.RegisterTransitCallback(cost_callback)
        routing.SetArcCostEvaluatorOfVehicle(idx, v)
        transit_callback_indices.append(idx)

    # 2. Capacity
    def demand_callback(from_index):
        return data["demands"][manager.IndexToNode(from_index)]
    demand_callback_index = routing.RegisterUnaryTransitCallback(demand_callback)
    routing.AddDimensionWithVehicleCapacity(demand_callback_index, 0, data["vehicle_capacities"], Tr

    # 3. Urban Exclusion Zone
    allowed_truck_nodes = ["Entrepôt Crissier", "Alloboissons", "Migros Ouchy", "The Lacustre", "Jet
    for i in range(len(data["node_names"])):
        if data["node_names"][i] not in allowed_truck_nodes:
```

```

routing.VehicleVar(manager.NodeToIndex(i)).RemoveValue(4)

# 4. Forced Lakeside Allocation
for node_name in ["Migros Ouchy", "The Lacustre", "Jetée de la compagnie", "La rincette"]:
    if node_name in data["node_names"]:
        routing.VehicleVar(manager.NodeToIndex(data["node_names"].index(node_name))).SetValue(4)

# 5. Forced Sequence: Givisiez (Alloboissons) First
if "Alloboissons" in data["node_names"]:
    allob_idx = manager.NodeToIndex(data["node_names"].index("Alloboissons"))
    routing.solver().Add(routing.NextVar(routing.Start(4)) == allob_idx)
    str: url_distances

# Solve View
search_parameters = pywrapcp.DefaultRoutingSearchParameters()
search_parameters.first_solution_strategy = routing_enums_pb2.FirstSolutionStrategy.PARALLEL_CHE
search_parameters.time_limit.FromSeconds(2) # Short limit for faster looping
solution = routing.SolveWithParameters(search_parameters)

if solution:
    cost = solution.ObjectiveValue() / 100.0
    # Count vehicles that actually leave the depot (ignore empty Tour 2s)
    active_vehicles = sum(1 for v in range(data["num_vehicles"]) if not routing.IsEnd(solution.V.
    return cost, active_vehicles
return None, None

# --- RUN THE ANALYSIS ---
# Simulating growth from +0% up to +60%
growth_multipliers = [1.0, 1.10, 1.20, 1.30, 1.40, 1.50, 1.60]
costs = []
active_counts = []
valid_multipliers = []

print("Running Sensitivity Analysis on Scenario 5... This will take about 10-15 seconds.")
for m in growth_multipliers:
    cost, count = evaluate_demand_scenario(m)
    if cost is not None:
        costs.append(cost)
        active_counts.append(count)
        valid_multipliers.append(m)
        print(f" -> +{int((m-1)*100)}% Demand: {cost:.2f} CHF | {count} Vehicles Used")
    else:
        print(f" -> +{int((m-1)*100)}% Demand: INFEASIBLE (Capacity Exceeded)")

# --- PLOT THE RESULTS ---
fig, ax1 = plt.subplots(figsize=(10, 6))

color1 = 'tab:blue'
ax1.set_xlabel('Demand Growth Multiplier (1.0 = Current 2025 Demand)', fontsize=12)
ax1.set_ylabel('Total Weekly Routing Cost (CHF)', color=color1, fontsize=12, fontweight='bold')
ax1.plot(valid_multipliers, costs, marker='o', color=color1, linewidth=2.5, markersize=8, label='Wee
ax1.tick_params(axis='y', labelcolor=color1)
ax1.grid(True, linestyle='--', alpha=0.6)

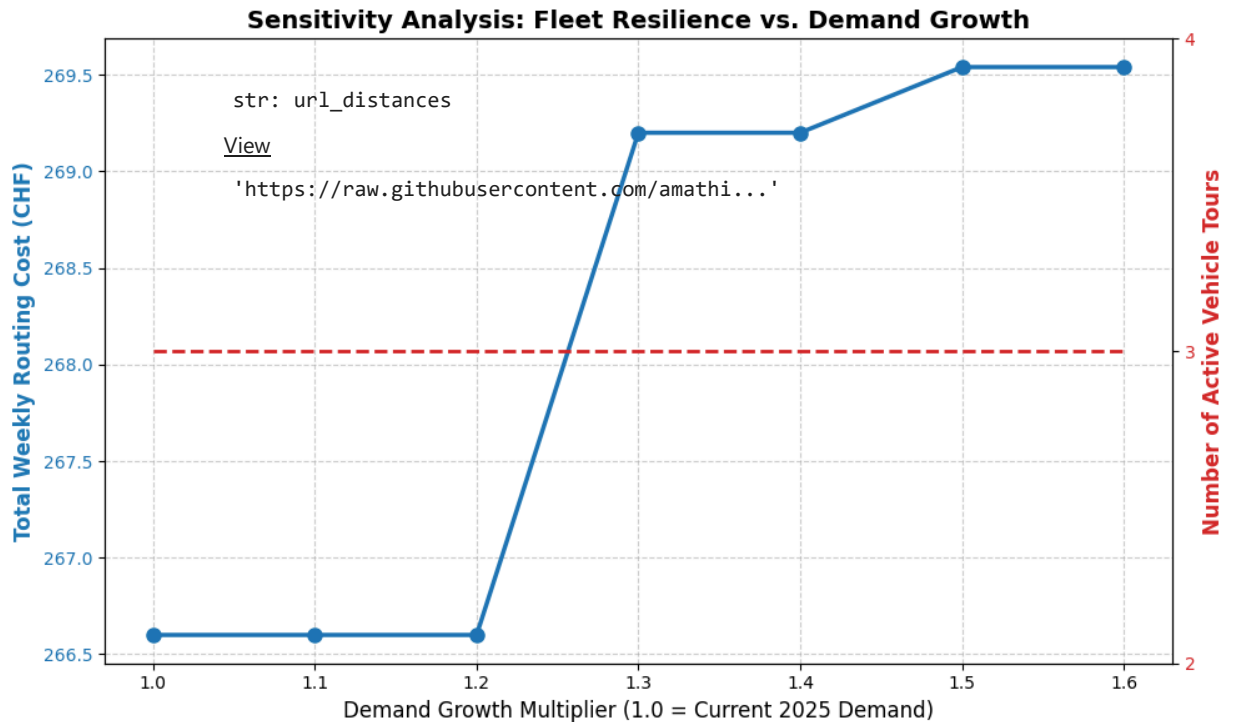
# Create a second y-axis for the vehicle count
ax2 = ax1.twinx()
color2 = 'tab:red'
ax2.set_ylabel('Number of Active Vehicle Tours', color=color2, fontsize=12, fontweight='bold')
ax2.step(valid_multipliers, active_counts, where='post', color=color2, linestyle='--', linewidth=2,
ax2.tick_params(axis='y', labelcolor=color2)
ax2.set_yticks(range(min(active_counts)-1, max(active_counts)+2))

plt.title('Sensitivity Analysis: Fleet Resilience vs. Demand Growth', fontsize=14, fontweight='bold')
fig.tight_layout()
plt.show()

```

Running Sensitivity Analysis on Scenario 5... This will take about 10-15 seconds.

-> +0% Demand: 266.60 CHF | 3 Vehicles Used
 -> +10% Demand: 266.60 CHF | 3 Vehicles Used
 -> +19% Demand: 266.60 CHF | 3 Vehicles Used
 -> +30% Demand: 269.20 CHF | 3 Vehicles Used
 -> +39% Demand: 269.20 CHF | 3 Vehicles Used
 -> +50% Demand: 269.54 CHF | 3 Vehicles Used
 -> +60% Demand: 269.54 CHF | 3 Vehicles Used



✓ Vehicle Operating Costs (Urban Tolling)

```
def evaluate_cost_scenario(transit_cost_cents):
    """Runs the Flexible VRP varying the cost of the Ford Transit."""
    data = create_flexible_data_model()

    # 1. UPDATE VEHICLE COSTS DYNAMICALLY
    # Base: Caddy=60, Transit=variable, Truck=150
    data["vehicle_costs_cents"] = [60, 60, transit_cost_cents, transit_cost_cents, 150]

    manager = pywrapcp.RoutingIndexManager(len(data["distance_matrix"]), data["num_vehicles"], data[
        routing = pywrapcp.RoutingModel(manager)

    # 2. Costs
    transit_callback_indices = []
    for v in range(data["num_vehicles"]):
        def cost_callback(from_index, to_index, vehicle_id=v):
            from_node = manager.IndexToNode(from_index)
            to_node = manager.IndexToNode(to_index)
            dist_m = data["distance_matrix"][from_node][to_node]
            return int((dist_m * data["vehicle_costs_cents"][vehicle_id]) / 1000)

        idx = routing.RegisterTransitCallback(cost_callback)
        routing.SetArcCostEvaluatorOfVehicle(idx, v)
        transit_callback_indices.append(idx)

    # 3. Capacity
    def demand_callback(from_index):
        return data["demands"][manager.IndexToNode(from_index)]
    demand_callback_index = routing.RegisterUnaryTransitCallback(demand_callback)
    routing.AddDimensionWithVehicleCapacity(demand_callback_index, 0, data["vehicle_capacities"], Tr
```

```

# 4. Urban Exclusion Zone (City Center Banned for Truck)
allowed_truck_nodes = ["Entrepôt Crissier", "Alloboissons", "Migros Ouchy", "The Lacustre", "Jet
for i in range(len(data["node_names"])):
    if data["node_names"][i] not in allowed_truck_nodes:
        routing.VehicleVar(manager.NodeToIndex(i)).RemoveValue(4)

# NOTE: We DO NOT force the lakeside nodes here. We want the algorithm to choose based on price!

# 5. Forced Sequence: Givisiez (Alloboissons) First
if "Alloboissons" in data["node_names"]:
    allob_idx = manager.NodeToIndex(data["node_names"].index("Alloboissons"))
    routing.solver().Add(routing.NextVar(routing.Start(4)) == allob_idx)
    View

# Solve
search_parameters = https://raw.githubusercontent.com/amathie/pywrapcp/DefaultRoutingSearchParameters()
search_parameters.first_solution_strategy = routing_enums_pb2.FirstSolutionStrategy.PARALLEL_CHE
search_parameters.time_limit.FromSeconds(2)
solution = routing.SolveWithParameters(search_parameters)

if solution:
    cost = solution.ObjectiveValue() / 100.0

    # Calculate how much total load was assigned to the Truck (Vehicle 4)
    truck_load = 0
    index = routing.Start(4)
    while not routing.IsEnd(index):
        node_index = manager.IndexToNode(index)
        truck_load += data["demands"][node_index]
        index = solution.Value(routing.NextVar(index))

    return cost, truck_load
return None, None

# --- RUN THE ANALYSIS ---
# Simulating Ford Transit costs from 0.80 CHF/km to 2.00 CHF/km
transit_costs_test = [80, 90, 100, 110, 120, 140, 160, 180, 200]
total_costs = []
truck_loads = []
valid_costs = []

print("Running Cost Sensitivity Analysis... Please wait.")
for cost_cents in transit_costs_test:
    total_cost, truck_load = evaluate_cost_scenario(cost_cents)
    if total_cost is not None:
        total_costs.append(total_cost)
        truck_loads.append(truck_load)
        valid_costs.append(cost_cents / 100.0) # Convert to CHF for X-axis
    print(f" -> Transit @ {cost_cents/100:.2f} CHF/km: Total Cost = {total_cost:.2f} CHF | Truck

# --- PLOT THE RESULTS ---
fig, ax1 = plt.subplots(figsize=(10, 6))

color1 = 'tab:green'
ax1.set_xlabel('Ford Transit Operating Cost (CHF / km)', fontsize=12)
ax1.set_ylabel('Total Weekly Routing Cost (CHF)', color=color1, fontsize=12, fontweight='bold')
ax1.plot(valid_costs, total_costs, marker='s', color=color1, linewidth=2.5, markersize=8, label='Wee
ax1.tick_params(axis='y', labelcolor=color1)
ax1.grid(True, linestyle='--', alpha=0.6)

# Create a second y-axis for the Truck Load
ax2 = ax1.twinx()
color2 = 'tab:purple'
ax2.set_ylabel('Total Load Assigned to the Heavy Truck (kg)', color=color2, fontsize=12, fontweight=
ax2.step(valid_costs, truck_loads, where='post', color=color2, linestyle='-', linewidth=3, label='Tr
ax2.tick_params(axis='y', labelcolor=color2)
ax2.set_ylim(5000, 6500) # Zoom in on the relevant load range

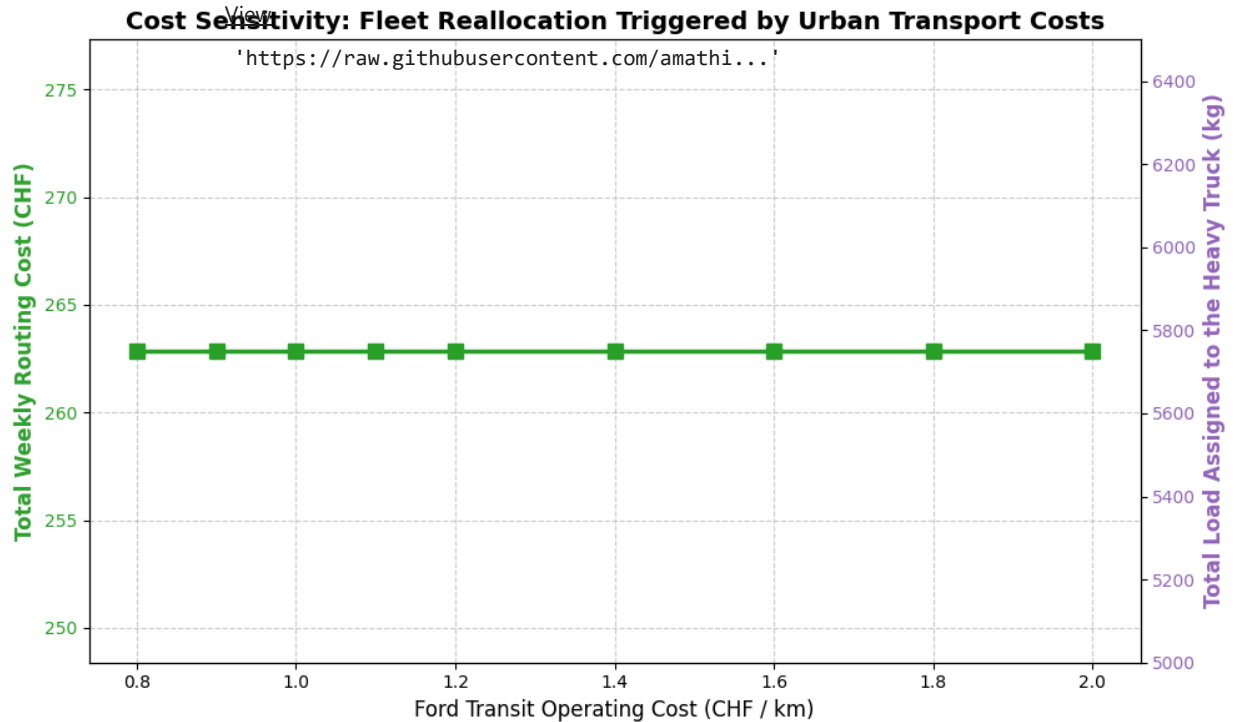
plt.title('Cost Sensitivity: Fleet Reallocation Triggered by Urban Transport Costs', fontsize=14, fo

```

```
fig.tight_layout()
plt.show()
```

Running Cost Sensitivity Analysis... Please wait.

```
-> Transit @ 0.80 CHF/km: Total Cost = 262.85 CHF | Truck Load = 1641 kg
-> Transit @ 0.90 CHF/km: Total Cost = 262.85 CHF | Truck Load = 1641 kg
-> Transit @ 1.00 CHF/km: Total Cost = 262.85 CHF | Truck Load = 1641 kg
-> Transit @ 1.10 CHF/km: Total Cost = 262.85 CHF | Truck Load = 1641 kg
-> Transit @ 1.20 CHF/km: Total Cost = 262.85 CHF | Truck Load = 1641 kg
-> Transit @ 1.40 CHF/km: Total Cost = 262.85 CHF | Truck Load = 1641 kg
-> Transit @ 1.60 CHF/km: Total Cost = 262.85 CHF | Truck Load = 1641 kg
-> Transit @ 1.80 CHF/km: Total Cost = 262.85 CHF | Truck Load = 1641 kg
-> Transit @ 2.00 CHF/km: Total Cost = 262.85 CHF | Truck Load = 1641 kg
```



✓ Environmental Impact & CO2 Emissions Analysis

```
def plot_co2_analysis():
    """
    CO2 analysis using REAL distances from our OR-Tools solver outputs.
    """

    # — Real distances from solver outputs (km) —————
    # S1: Unrestricted (Truck does almost everything)
    s1_distances = [0.0, 0.0, 160.55]
    # S2: Flexible Allocation
    s2_distances = [15.48, 20.40, 160.55]
    # S5: Final Model (Forced Lakeside + Forced Sequence)
    s5_distances = [12.79, 18.98, 164.25]

    co2_factors = [158, 195, 650]      # g/km per vehicle
    vehicle_names = ["VW Caddy", "Ford Transit", "Truck"]
    colors = ["#43A047", "#FB8C00", "#E53935"]
    scenario_colors = ["#5C6BC0", "#26A69A", "#AB47BC"]

    def calc_co2(distances):
        return [(d * f) / 1000 for d, f in zip(distances, co2_factors)]

    s1_co2 = calc_co2(s1_distances)
    s2_co2 = calc_co2(s2_distances)
    s5_co2 = calc_co2(s5_distances)
```

```

fig = plt.figure(figsize=(18, 10))
fig.patch.set_facecolor("#F8F9FA")

# — Graph 1 (top-left): CO2 per vehicle for each scenario —————
ax1 = fig.add_subplot(2, 2, 1)
x = np.arange(len(vehicle_names))
w = 0.25
b1 = ax1.bar(x - w, s1_co2, w, label="S1: Unrestricted", color=scenario_colors[0], alpha=0.85)
b2 = ax1.bar(x, s2_co2, w, label="S2: Flexible", color=scenario_colors[1], alpha=0.85)
b3 = ax1.bar(x + w, s5_co2, w, label="S5: Operational", color=scenario_colors[2], alpha=0.85)

for bars in [b1, b2, b3]:
    for bar in bars:
        h = bar.get_height()
        if h > 0.01:
            ax1.text(bar.get_x() + bar.get_width()/2, h + 1.0,
                    f"{h:.1f}", ha="center", va="bottom", fontsize=9, fontweight="bold")

ax1.set_title("CO2 per Vehicle – Scenario Comparison", fontsize=12, fontweight="bold")
ax1.set_ylabel("CO2 Emitted (kg)")
ax1.set_xticks(x)
ax1.set_xticklabels(vehicle_names)
ax1.legend(fontsize=9)
ax1.grid(axis="y", linestyle="--", alpha=0.5)
ax1.set_facecolor("#FAFAFA")

# — Graph 2 (top-right): Total CO2 per scenario —————
ax2 = fig.add_subplot(2, 2, 2)
totals = [sum(s1_co2), sum(s2_co2), sum(s5_co2)]
scenario_labels = ["Scenario 1\n(Unrestricted)", "Scenario 2\n(Flexible)", "Scenario 5\n(Operational)"]
bars = ax2.bar(scenario_labels, totals, color=scenario_colors, edgecolor="white", linewidth=1.5)

for bar, val in zip(bars, totals):
    ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1.0,
            f"{val:.1f} kg CO2", ha="center", va="bottom", fontsize=10, fontweight="bold")

# Highlight best scenario
best_idx = totals.index(min(totals))
bars[best_idx].set_edgecolor("#FFD700")
bars[best_idx].set_linewidth(3)
ax2.set_title("Total CO2 Emissions per Scenario", fontsize=12, fontweight="bold")
ax2.set_ylabel("Total CO2 (kg)")
ax2.set_ylim(0, max(totals) * 1.15)
ax2.grid(axis="y", linestyle="--", alpha=0.5)
ax2.set_facecolor("#FAFAFA")
ax2.text(0.5, 0.95, "★ = Best Scenario", transform=ax2.transAxes,
        ha="center", fontsize=9, color="gray", style="italic")

# — Graph 3 (bottom-left): Pie Chart CO2 Scenario 3 —————
ax3 = fig.add_subplot(2, 2, 3)

labels_pie = []
values_pie = []
colors_pie = []
for name, val, col in zip(vehicle_names, s5_co2, colors):
    if val > 0.01:
        labels_pie.append(f"{name}\n{val:.1f} kg")
        values_pie.append(val)
        colors_pie.append(col)

wedges, texts, autotexts = ax3.pie(
    values_pie,
    labels=labels_pie,
    colors=colors_pie,
    autopct='%1.1f%%',
    startangle=140,
    wedgeprops={'edgecolor': 'white', 'linewidth': 2},
    pctdistance=0.75,
    textprops={'fontsize': 10})

```



```

)
for autotext in autotexts:
    autotext.set_fontweight('bold')

ax3.set_title(f"C02 Breakdown – Final Operational Model\nTotal: {sum(s5_co2):.1f} kg C02",
              fontsize=12, fontweight="bold")

# — Graph 4 (bottom-right): C02 efficiency (g C02 per kg delivered) —
ax4 = fig.add_subplot(2, 2, 4)
total_load = 7436 # UPDATED: Real total load from the final VRP outputs
efficiency = [t / total_load * 1000 for t in totals] # g C02 per kg delivered
str: url_distances

bars4 = ax4.bars(scenario_labels, efficiency, color=scenario_colors, edgecolor="white", linewidth=1)
for bar, val in zip(bars4, efficiency):
    ax4.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.2,
             f"{val:.1f} g/kg", ha="center", va="bottom", fontsize=10, fontweight="bold")

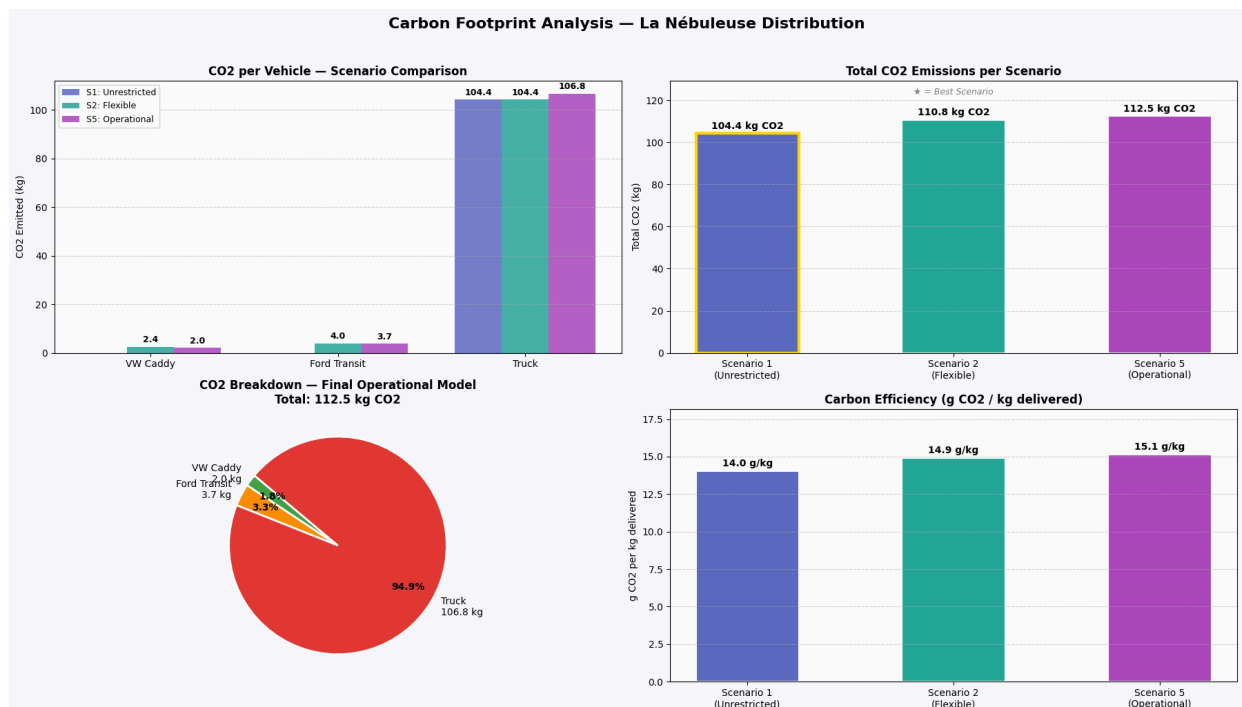
ax4.set_title("Carbon Efficiency (g C02 / kg delivered)", fontsize=12, fontweight="bold")
ax4.set_ylabel("g C02 per kg delivered")
ax4.set_ylim(0, max(efficiency) * 1.2)
ax4.grid(axis="y", linestyle="--", alpha=0.5)
ax4.set_facecolor("#FAFAFA")

plt.suptitle("Carbon Footprint Analysis – La Nébuleuse Distribution",
            fontsize=16, fontweight="bold", y=1.02)
plt.tight_layout()
plt.show()

print(f"\n--- C02 SUMMARY ---")
print(f" Scenario 1 (Unrestricted) : {sum(s1_co2):.2f} kg C02 ({sum(s1_co2)/total_load*1000:.1f} g/kg delivered)")
print(f" Scenario 2 (Flexible)      : {sum(s2_co2):.2f} kg C02 ({sum(s2_co2)/total_load*1000:.1f} g/kg delivered)")
print(f" Scenario 3 (Operational)     : {sum(s5_co2):.2f} kg C02 ({sum(s5_co2)/total_load*1000:.1f} g/kg delivered)")

plot_co2_analysis()

```



--- C02 SUMMARY ---

Scenario 1 (Unrestricted) : 104.36 kg C02 (14.0 g/kg delivered)
 Scenario 2 (Flexible) : 110.78 kg C02 (14.9 g/kg delivered)
 Scenario 3 (Operational) : 112.48 kg C02 (15.1 g/kg delivered)

```

# =====
# 1. DATA PREPARATION & PACKAGING PARAMETERS
# =====

# Production Plant (Node 0)
PLANT = {"name": "Brasserie Renens", "coords": (46.5367, 6.5866)}

# Potential Hub Locations (J)
# Capacities in Pallets, Fixed Costs in CHF/week
HUBS = {
    "Crissier": {"coords": (46.5492, 6.5786), "fixed_cost": 1200, "capacity_pallets": 50},
    "Renens_Expansion": {"coords": (46.5367, 6.5866), "fixed_cost": 1000, "capacity_pallets": 50},
    "Givisiez": {"coords": (46.8152, 7.1275), "fixed_cost": 1200, "capacity_pallets": 40}
}

# Customers (C)
# Demands in Pallets
CUSTOMERS = {
    "Urban_HORECA": {"coords": (46.5210, 6.6300), "pallets": 4.5},
    "Alloboissons": {"coords": (46.8152, 7.1275), "pallets": 11.0}
}

AVG_GROSS_WEIGHT_PER_PALLET = 600 # kg (Net beer + Tare weight of packaging)

# =====
# 2. DISTANCE & TRANSPORT COST FUNCTIONS
# =====

def haversine(c1, c2):
    """Calculates straight-line distance in km between two GPS coordinates."""
    R = 6371.0
    lat1, lon1, lat2, lon2 = map(math.radians, [c1[0], c1[1], c2[0], c2[1]])
    a = math.sin((lat2-lat1)/2)**2 + math.cos(lat1)*math.cos(lat2)*math.sin((lon2-lon1)/2)**2
    return R * (2 * math.atan2(math.sqrt(a), math.sqrt(1 - a)))

# Transport Costs (CHF per GROSS kg per km) based on vehicle payload efficiencies
COST_PER_KG_KM_TIER1 = 1.50 / 8300 # Heavy Truck for Plant -> Hub shuttling
COST_PER_KG_KM_TIER2_URBAN = 0.80 / 1200 # Van for Hub -> Urban delivery
COST_PER_KG_KM_TIER2_BULK = 1.50 / 8300 # Truck for Hub -> Bulk delivery

# =====
# 3. MILP SOLVER ENGINE
# =====

def evaluate_infrastructure_scenario(scenario_name, active_hubs):
    """
    Builds and solves the MILP model for a specific infrastructure scenario.
    active_hubs: list of hub names that are FORCED to be open.
    """
    solver = pywraplp.Solver.CreateSolver('SCIP')
    if not solver:
        return None

    # --- Variables ---
    Y, X1, X2 = {}, {}, {}

    for j in HUBS:
        # Force the Y variable to 1 if it's in the scenario, 0 otherwise
        lb = 1 if j in active_hubs else 0
        ub = 1 if j in active_hubs else 0
        Y[j] = solver.IntVar(lb, ub, f'Y_{j}')
        X1[j] = solver.NumVar(0, solver.infinity(), f'X1_{j}')
```

```

for c, c_data in CUSTOMERS.items():
    solver.Add(sum(X2[j, c] for j in HUBS) == c_data["pallets"])

# 2. Flow Conservation: Volume entering hub = Volume leaving hub
for j in HUBS:
    solver.Add(X1[j] == sum(X2[j, c] for c in CUSTOMERS))

# 3. Hub Capacity: Cannot exceed pallet capacity, and forces flow to 0 if hub is closed
for j, j_data in HUBS.items():
    solver.Add(X1[j] <= j_data["capacity_pallets"] * Y[j])

# --- Objective Function ---
objective = solver.Objective()

for j, j_data in HUBS.items():
    # Add Fixed Facility Costs
    objective.SetCoefficient(Y[j], j_data["fixed_cost"])

    # Add Tier 1 Transport Costs (Plant -> Hub)
    dist_t1 = haversine(PLANT["coords"], j_data["coords"])
    cost_t1_per_pallet = dist_t1 * COST_PER_KG_KM_TIER1 * AVG_GROSS_WEIGHT_PER_PALLET
    objective.SetCoefficient(X1[j], cost_t1_per_pallet)

    # Add Tier 2 Transport Costs (Hub -> Customer)
    for c, c_data in CUSTOMERS.items():
        dist_t2 = haversine(j_data["coords"], c_data["coords"])
        rate = COST_PER_KG_KM_TIER2_URBAN if c == "Urban_HORECA" else COST_PER_KG_KM_TIER2_BULK
        cost_t2_per_pallet = dist_t2 * rate * AVG_GROSS_WEIGHT_PER_PALLET
        objective.SetCoefficient(X2[j, c], cost_t2_per_pallet)

objective.SetMinimization()

# Solve
status = solver.Solve()

if status == pywraplp.Solver.OPTIMAL:
    return objective.Value()
else:
    return None

# =====
# 4. EXECUTE SCENARIOS & VISUALIZE
# =====

print("Solving Multi-Echelon CFLP Scenarios...\n")

# Define the hubs that are open in each scenario
scenario_results = {
    "Scenario 1:\nBaseline (Crissier)": evaluate_infrastructure_scenario("S1", ["Crissier"]),
    "Scenario 2:\nUnified (Renens)": evaluate_infrastructure_scenario("S2", ["Renens_Expansion"]),
    "Scenario 3:\nDual Hubs (Crissier + Givisiez)": evaluate_infrastructure_scenario("S3", ["Crissie"])
}

print("--- WEEKLY TOTAL SUPPLY CHAIN COSTS ---")
for name, cost in scenario_results.items():
    print(f"{name.replace('\n', ' ')}: {cost:,.2f} CHF")

# Plotting the Results
fig, ax = plt.subplots(figsize=(10, 6))
colors = ['#E57373', '#81C784', '#64B5F6']
bars = ax.bar(scenario_results.keys(), scenario_results.values(), color=colors, edgecolor='black', 1

ax.set_title("Strategic Network Optimization: Total Weekly Cost per Scenario", fontsize=14, fontweig
ax.set_ylabel("Total Supply Chain Cost (CHF/week)", fontsize=12)
ax.set_ylim(0, max(scenario_results.values()) * 1.2)

# Add value labels on top of the bars
for bar in bars:
    yval = bar.get_height()

```

```

ax.text(bar.get_x() + bar.get_width()/2, yval + 50, f"{yval:,.2f} CHF", ha='center', va='bottom')

plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()

```

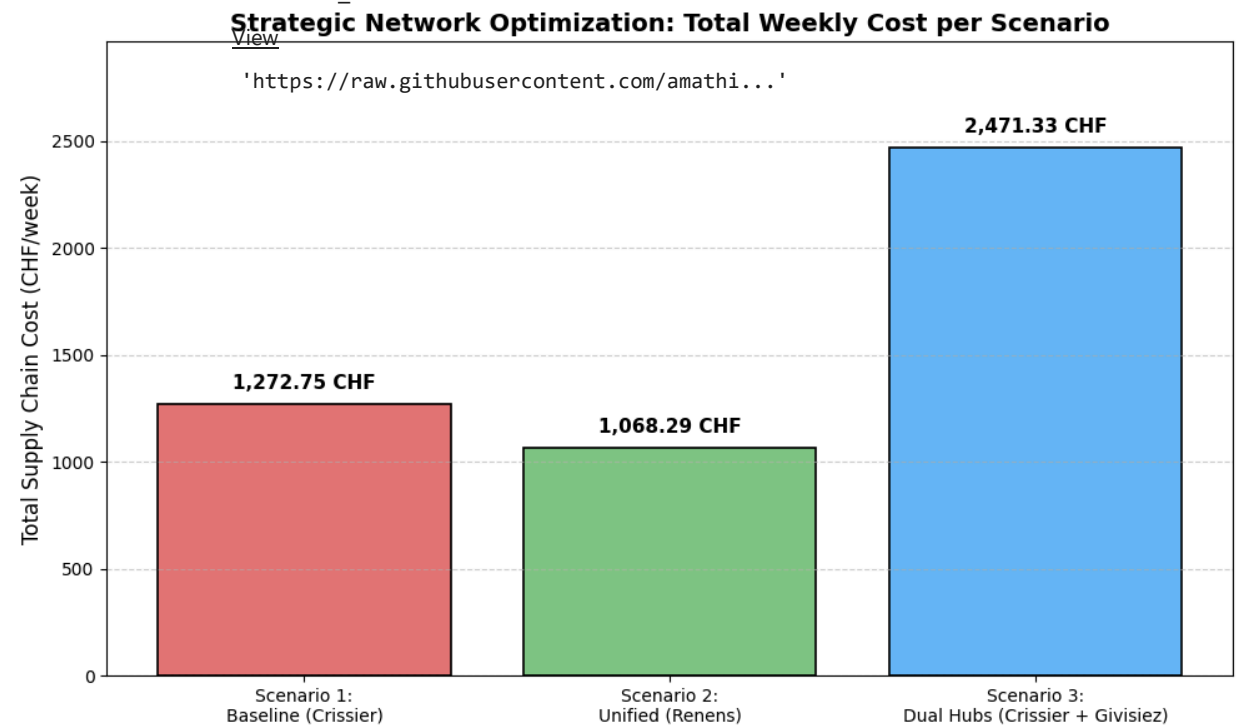
Solving Multi-Echelon CFLP Scenarios...

--- WEEKLY TOTAL SUPPLY CHAIN COSTS ---

Scenario 1: Baseline (Crissier): 1,272.75 CHF

Scenario 2: Unified (Renens): 1,068.29 CHF

Scenario 3: Dual Hubs (Crissier + Givisiez): 2,471.33 CHF



✓ 7. Electric Vehicle (EV) Fleet Analysis

```

# =====
# EV ANALYSIS – VEHICLE PARAMETER DICTS
# =====

# Weekly km from Scenario 3 VRP solver outputs (Section 6.3)
WEEKS_PER_YEAR = 52

VEHICLES = {
    "VW Caddy": {
        "weekly_km": 12.79,
        "diesel": {
            "capex": 32_000,
            "fuel_l_per_100km": 6.5,
            "maintenance_chf_per_km": 0.08,
            "residual_pct": 0.15,
        },
        "ev": {
            "capex": 45_000,
            "kwh_per_100km": 19.0,
            "maintenance_chf_per_km": 0.05,
            "residual_pct": 0.15,
            "battery_kwh": 55,
        },
    },
    "Ford Transit": {
        "weekly_km": 18.98,

```

```

        "diesel": {
            "capex": 42_000,
            "fuel_l_per_100km": 8.5,
            "maintenance_chf_per_km": 0.10,
            "residual_pct": 0.15,
        },
        "ev": {
            "capex": 65_000,
            "kwh_per_100km": 25.0,
            "maintenance_chf_per_km": 0.06,
            "residual_pct": 0.15,
            "battery_kwh": 75,
        },
    },
    'https://raw.githubusercontent.com/amathie...'
}

TRUCK = {
    "weekly_km": 164.25,
    "diesel": {
        "capex": 85_000,
        "fuel_l_per_100km": 28.0,
        "maintenance_chf_per_km": 0.18,
        "residual_pct": 0.15,
    },
    "ev": {
        "capex": 350_000,
        "kwh_per_100km": 105.0,
        "maintenance_chf_per_km": 0.12,
        "residual_pct": 0.15,
        "battery_kwh": 336,
    },
}

PARAMS = {
    "diesel_chf_per_l": 1.85,
    "elec_chf_per_kwh": 0.25,
    "diesel_co2_kg_per_l": 2.68,
    "grid_co2_g_per_kwh": 30.0,
    "battery_co2_kg_per_kwh": 70.0, # Li-ion mfg LCA (Ricardo/IVL 2023, mid-range)
    "horizon_years": 10,
    "discount_rate": 0.05, # NPV discount rate
}

print("EV analysis parameters loaded.")
print(f"Annual km – Caddy: {VEHICLES['VW Caddy']['weekly_km']*WEEKS_PER_YEAR:.0f} | "
      f"Transit: {VEHICLES['Ford Transit']['weekly_km']*WEEKS_PER_YEAR:.0f} | "
      f"Truck: {TRUCK['weekly_km']*WEEKS_PER_YEAR:.0f}")
print(f"Battery embedded CO2: {PARAMS['battery_co2_kg_per_kwh']} kg CO2/kWh")
print(f"  Caddy battery debt   : {VEHICLES['VW Caddy']['ev']['battery_kwh'] * PARAMS['battery_co2_kg_per_kwh']} kg CO2")
print(f"  Transit battery debt  : {VEHICLES['Ford Transit']['ev']['battery_kwh'] * PARAMS['battery_co2_kg_per_kwh']} kg CO2")
print(f"  Truck battery debt    : {TRUCK['ev']['battery_kwh'] * PARAMS['battery_co2_kg_per_kwh']} kg CO2")

```

```

EV analysis parameters loaded.
Annual km – Caddy: 665 | Transit: 987 | Truck: 8541
Battery embedded CO2: 70.0 kg CO2/kWh
  Caddy battery debt   : 3,850 kg CO2
  Transit battery debt  : 5,250 kg CO2
  Truck battery debt    : 23,520 kg CO2

```

```

# =====
# TCO MODEL (no residual in curve, NPV variant)
# =====

def annual_opex(vehicle_data, fuel_type, params, override_km_year=None,
                override_diesel_price=None, override_elec_price=None):
    """Energy + maintenance for one year, in CHF."""
    annual_km = override_km_year if override_km_year is not None else vehicle_data["weekly_km"] * WE
    spec = vehicle_data[fuel_type]

```

```

if fuel_type == "diesel":
    price = override_diesel_price if override_diesel_price is not None else params["diesel_chf_p
    energy = (spec["fuel_l_per_100km"] / 100.0) * annual_km * price
else:
    price = override_elec_price if override_elec_price is not None else params["elec_chf_per_kwh
    energy = (spec["kwh_per_100km"] / 100.0) * annual_km * price

maint = spec["maintenance_chf_per_km"] * annual_km
return energy + maint

str: url_distances

def compute_tco_curve(vehicle_data, fuel_type, params, override_km_year=None):
    """Cumulative CHF cost from year 0 (CAPEX only) to horizon.
    https://raw.githubusercontent.com/amathi...

    Residual value is NOT applied inside the curve (would create artificial drop).
    Use compute_residual() separately if you want resale credit at terminal year.
    """

    horizon = params["horizon_years"]
    spec = vehicle_data[fuel_type]
    yearly = annual_opex(vehicle_data, fuel_type, params, override_km_year=override_km_year)
    cumulative = [spec["capex"]]
    for _ in range(horizon):
        cumulative.append(cumulative[-1] + yearly)
    return cumulative

def compute_npv_tco_curve(vehicle_data, fuel_type, params, override_km_year=None):
    """Cumulative present-value CHF. CAPEX at year 0 (no discount), opex discounted."""
    horizon = params["horizon_years"]
    r = params["discount_rate"]
    spec = vehicle_data[fuel_type]
    yearly = annual_opex(vehicle_data, fuel_type, params, override_km_year=override_km_year)
    cumulative = [spec["capex"]]
    for year in range(1, horizon + 1):
        pv = yearly / ((1 + r) ** year)
        cumulative.append(cumulative[-1] + pv)
    return cumulative

def compute_residual(vehicle_data, fuel_type):
    spec = vehicle_data[fuel_type]
    return spec["capex"] * spec["residual_pct"]

def find_breakeven_year(diesel_curve, ev_curve):
    for year, (d, e) in enumerate(zip(diesel_curve, ev_curve)):
        if year == 0:
            continue
        if e <= d:
            return year
    return None

# Sanity check
caddy_d = compute_tco_curve(VEHICLES["VW Caddy"], "diesel", PARAMS)
caddy_e = compute_tco_curve(VEHICLES["VW Caddy"], "ev", PARAMS)
print(f"VW Caddy 10y TCO (no residual) – Diesel: {caddy_d[-1]:,.0f} CHF | EV: {caddy_e[-1]:,.0f} CHF
print(f"  Resale credit at year 10: Diesel {compute_residual(VEHICLES['VW Caddy'],'diesel'):.0f} CHF
      f"EV {compute_residual(VEHICLES['VW Caddy'],'ev'):.0f} CHF")
print(f"  Break-even (operational): {find_breakeven_year(caddy_d, caddy_e)}")

VW Caddy 10y TCO (no residual) – Diesel: 33,332 CHF | EV: 45,648 CHF
Resale credit at year 10: Diesel 4,800 CHF | EV 6,750 CHF
Break-even (operational): None

# =====
# BREAK-EVEN VISUALIZATION (VANS)

```

```
# =====

def plot_tco_breakeven():
    fig, axes = plt.subplots(1, 2, figsize=(15, 6))
    fig.patch.set_facecolor("#F8F9FA")

    horizon = PARAMS["horizon_years"]
    years = list(range(horizon + 1))

    for ax, (name, vdata) in zip(axes, VEHICLES.items()):
        diesel_curve = compute_tco_curve(vdata, "diesel", PARAMS)
        ev_curve = compute_tco_curve(vdata, "ev", PARAMS)
        be_year = find_breakeven_year(diesel_curve, ev_curve)

        ax.plot(years, diesel_curve, "-o", color="#E53935", label="Diesel", linewidth=2, markersize=5)
        ax.plot(years, ev_curve, "-o", color="#43A047", label="Electric", linewidth=2, markersize=5)

        if be_year is not None:
            ax.axvline(be_year, color="#1565C0", linestyle="--", alpha=0.7)
            ax.text(be_year + 0.1, max(diesel_curve) * 0.5,
                    f"Break-even\nYear {be_year}", color="#1565C0", fontweight="bold")
        else:
            ax.text(0.5, 0.95,
                    "No break-even within 10y horizon",
                    transform=ax.transAxes, ha="center", color="#C62828",
                    fontweight="bold", style="italic")

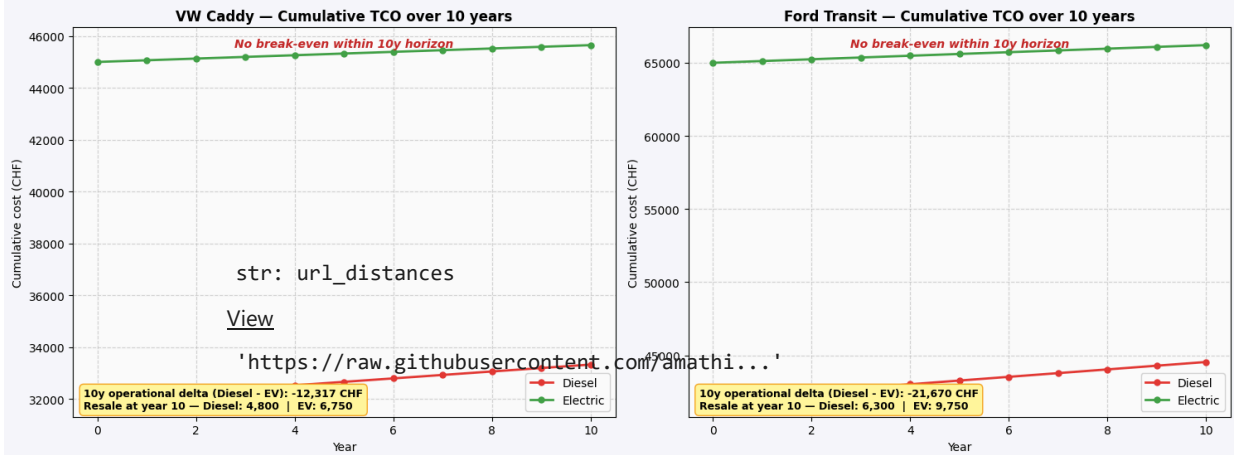
        delta = diesel_curve[-1] - ev_curve[-1]
        resale_d = compute_residual(vdata, "diesel")
        resale_e = compute_residual(vdata, "ev")
        ax.text(0.02, 0.02,
                f"10y operational delta (Diesel - EV): {delta:,.0f} CHF\n"
                f"Resale at year 10 - Diesel: {resale_d:,.0f} | EV: {resale_e:,.0f}",
                transform=ax.transAxes, fontsize=9, fontweight="bold",
                bbox=dict(boxstyle="round,pad=0.4", facecolor="#FFF59D", edgecolor="#F9A825"))

        ax.set_title(f"{name} - Cumulative TCO over 10 years", fontsize=12, fontweight="bold")
        ax.set_xlabel("Year")
        ax.set_ylabel("Cumulative cost (CHF)")
        ax.legend()
        ax.grid(linestyle="--", alpha=0.5)
        ax.set_facecolor("#FAFAFA")

    plt.suptitle("EV vs Diesel - TCO Break-Even Analysis (Urban Fleet, undiscounted)",
                fontsize=15, fontweight="bold", y=1.02)
    plt.tight_layout()
    plt.show()

plot_tco_breakeven()
```

EV vs Diesel — TCO Break-Even Analysis (Urban Fleet, undiscounted)



```
# =====
# LIFECYCLE CO2 – INCLUDING BATTERY MANUFACTURING
# =====

def compute_co2_curve(vehicle_data, fuel_type, params, include_battery=True):
    """Cumulative kg CO2. EV starts at battery manufacturing debt (year 0).
    Diesel starts at 0 (vehicle manufacturing CO2 ignored – comparable across both,
    so excluding it is a fair simplification)."""
    horizon = params["horizon_years"]
    annual_km = vehicle_data["weekly_km"] * WEEKS_PER_YEAR
    spec = vehicle_data[fuel_type]

    if fuel_type == "diesel":
        annual_kg = (spec["fuel_l_per_100km"] / 100.0) * annual_km * params["diesel_co2_kg_per_l"]
        start = 0.0
    else:
        annual_kg = (spec["kwh_per_100km"] / 100.0) * annual_km * (params["grid_co2_g_per_kwh"] / 10)
        start = spec["battery_kwh"] * params["battery_co2_kg_per_kwh"] if include_battery else 0.0

    cumulative = [start]
    for _ in range(horizon):
        cumulative.append(cumulative[-1] + annual_kg)
    return cumulative

def find_carbon_payback(diesel_curve, ev_curve):
    """First year EV cumulative CO2 <= diesel cumulative CO2."""
    for year, (d, e) in enumerate(zip(diesel_curve, ev_curve)):
        if year == 0:
            continue
        if e <= d:
            return year
    return None

def plot_co2_lifecycle():
    fig, axes = plt.subplots(1, 2, figsize=(15, 6))
    fig.patch.set_facecolor("#F8F9FA")

    horizon = PARAMS["horizon_years"]
    years = list(range(horizon + 1))

    total_diesel_kg = 0.0
    total_ev_kg = 0.0

    for ax, (name, vdata) in zip(axes, VEHICLES.items()):
        diesel_curve = compute_co2_curve(vdata, "diesel", PARAMS)
        ev_curve = compute_co2_curve(vdata, "ev", PARAMS, include_battery=True)
        cb_year = find_carbon_payback(diesel_curve, ev_curve)
```



```

ax.plot(years, diesel_curve, "-o", color="#E53935", label="Diesel (operational)", linewidth=2)
ax.plot(years, ev_curve, "-o", color="#43A047", label="EV (battery + grid)", linewidth=2, ma

# Fill regions: red where EV > diesel (carbon debt), green where EV < diesel (advantage)
diesel_arr = np.array(diesel_curve)
ev_arr = np.array(ev_curve)
ax.fill_between(years, diesel_arr, ev_arr, where=(ev_arr > diesel_arr),
                color="#FF8A65", alpha=0.35, label="EV carbon debt")
ax.fill_between(years, diesel_arr, ev_arr, where=(ev_arr <= diesel_arr),
                color="#A5D6A7", alpha=0.45, label="EV CO2 advantage")
str: url_distances

battery_debt = vdata["ev"]["battery_kwh"] * PARAMS["battery_co2_kg_per_kwh"]
saved = diesel_curve[-1] - ev_curve[-1]
total_diesel_kg += diesel_curve[-1]
total_ev_kg += ev_curve[-1]

cb_text = f"Carbon payback: year {cb_year}" if cb_year else "No carbon payback in 10y"
ax.text(0.02, 0.97,
        f"Battery debt at y0: {battery_debt:,.0f} kg CO2\n"
        f"{cb_text}\n"
        f"Net 10y CO2 saved: {saved:,.0f} kg ({saved / max(diesel_curve[-1], 1) * 100:.1f}%)")
transform=ax.transAxes, fontsize=9, fontweight="bold", va="top",
bbox=dict(boxstyle="round,pad=0.4", facecolor="#C8E6C9", edgecolor="#2E7D32"))

if cb_year is not None:
    ax.axvline(cb_year, color="#1565C0", linestyle="--", alpha=0.6)

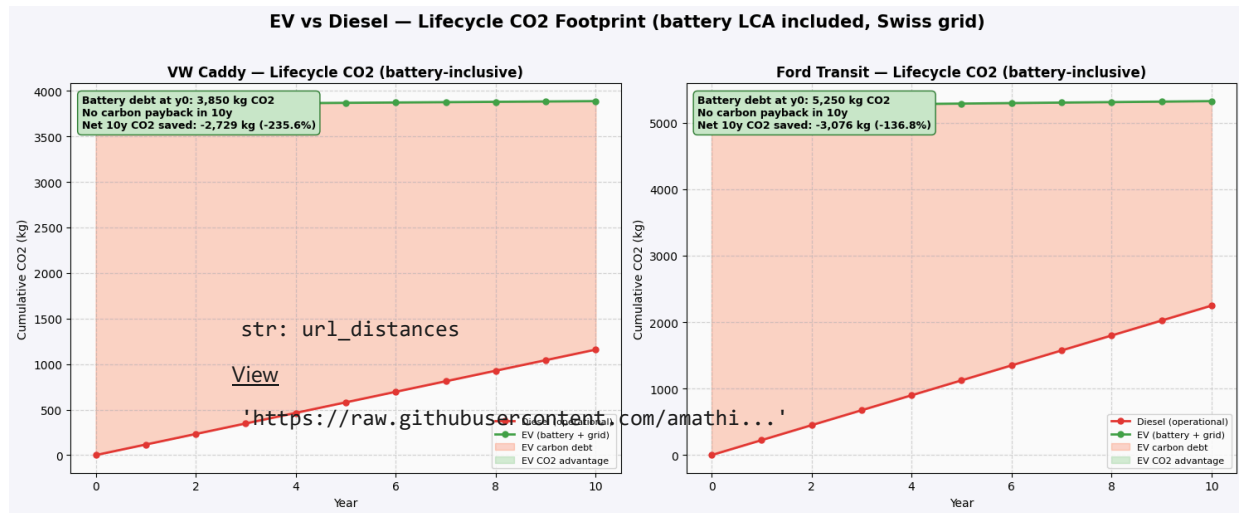
ax.set_title(f"{name} – Lifecycle CO2 (battery-inclusive)", fontsize=12, fontweight="bold")
ax.set_xlabel("Year")
ax.set_ylabel("Cumulative CO2 (kg)")
ax.legend(loc="lower right", fontsize=8)
ax.grid(linestyle="--", alpha=0.5)
ax.set_facecolor("#FAFAFA")

plt.suptitle("EV vs Diesel – Lifecycle CO2 Footprint (battery LCA included, Swiss grid)",
            fontsize=15, fontweight="bold", y=1.02)
plt.tight_layout()
plt.show()

print(f"\n--- 10-YEAR FLEET CO2 (vans only, battery LCA included) ---")
print(f" Diesel total : {total_diesel_kg:,.0f} kg CO2")
print(f" EV total      : {total_ev_kg:,.0f} kg CO2")
print(f" Net saved      : {total_diesel_kg - total_ev_kg:,.0f} kg CO2")
print(f"      ({(total_diesel_kg - total_ev_kg) / max(total_diesel_kg, 1) * 100:.1f}% reduction)")

plot_co2_lifecycle()

```



--- 10-YEAR FLEET CO2 (vans only, battery LCA included) ---

Diesel total : 3,407 kg CO2
 EV total : 9,212 kg CO2
 Net saved : -5,805 kg CO2 (-170.4% reduction)

```
# =====
# TRUCK ELECTRIFICATION – TCO + LIFECYCLE CO2
# =====

truck_diesel = compute_tco_curve(TRUCK, "diesel", PARAMS)
truck_ev = compute_tco_curve(TRUCK, "ev", PARAMS)
truck_be = find_breakeven_year(truck_diesel, truck_ev)

truck_co2_diesel = compute_co2_curve(TRUCK, "diesel", PARAMS)
truck_co2_ev = compute_co2_curve(TRUCK, "ev", PARAMS, include_battery=True)
truck_cb = find_carbon_payback(truck_co2_diesel, truck_co2_ev)

fig, axes = plt.subplots(1, 2, figsize=(15, 5.5))
fig.patch.set_facecolor("#F8F9FA")

years = list(range(PARAMS["horizon_years"] + 1))

ax = axes[0]
ax.plot(years, truck_diesel, "-o", color="#E53935", label="Diesel Truck", linewidth=2, markersize=5)
ax.plot(years, truck_ev, "-o", color="#43A047", label="eActros 300", linewidth=2, markersize=5)
ax.set_title("Heavy Truck – Cumulative TCO (10 years)", fontsize=12, fontweight="bold")
ax.set_xlabel("Year")
ax.set_ylabel("Cumulative cost (CHF)")
ax.legend()
ax.grid(linestyle="--", alpha=0.5)
ax.set_facecolor("#FAFAFA")

be_label = f"Break-even reached year {truck_be}" if truck_be else "No break-even within 10 years"
delta = truck_diesel[-1] - truck_ev[-1]
ax.text(0.02, 0.95,
       f"{be_label}\n10y delta (Diesel - EV): {delta:,.0f} CHF",
       transform=ax.transAxes, fontsize=10, fontweight="bold", va="top",
       bbox=dict(boxstyle="round,pad=0.4",
                 facecolor=("#FFCDD2" if (truck_be is None or delta < 0) else "#C8E6C9"),
                 edgecolor="#B71C1C"))

ax = axes[1]
diesel_arr = np.array(truck_co2_diesel)
ev_arr = np.array(truck_co2_ev)
ax.plot(years, truck_co2_diesel, "-o", color="#E53935", label="Diesel (operational)", linewidth=2, marker="o")
ax.plot(years, truck_co2_ev, "-o", color="#43A047", label="EV (battery + grid)", linewidth=2, marker="o")
ax.fill_between(years, diesel_arr, ev_arr, where=(ev_arr > diesel_arr),
               color="#FF8A65", alpha=0.35, label="EV carbon debt")
ax.fill_between(years, diesel_arr, ev_arr, where=(ev_arr <= diesel_arr),
```

```

color="#A5D6A7", alpha=0.45, label="EV CO2 advantage")

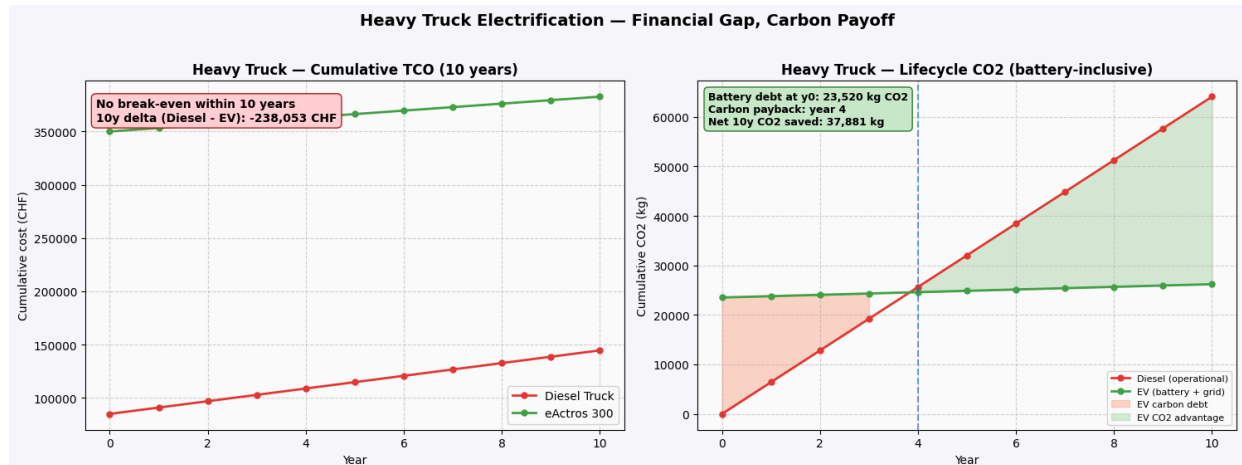
if truck_cb is not None:
    ax.axvline(truck_cb, color="#1565C0", linestyle="--", alpha=0.6)

battery_debt = TRUCK["ev"]["battery_kwh"] * PARAMS["battery_co2_kg_per_kwh"]
co2_saved = truck_co2_diesel[-1] - truck_co2_ev[-1]
cb_text = f"Carbon payback: year {truck_cb}" if truck_cb else "No carbon payback in 10y"
ax.set_title("Heavy Truck – Lifecycle CO2 (battery-inclusive)", fontsize=12, fontweight="bold")
ax.set_xlabel("Year")
ax.set_ylabel("Cumulative CO2 (kg)")
ax.legend(loc="lower right", fontsize=8)
ax.grid(linestyle='view', alpha=0.5)
ax.set_facecolor("#FAFAFA")
ax.text(0.02, 0.97, 'https://raw.githubusercontent.com/amathie5/20544258974c267d37f17c1409612784/sl_project_vf-ev.ipynb#scrollTo=32bSGrTX2kFY&print...')
    f"Battery debt at y0: {battery_debt:,.0f} kg CO2\n"
    f"{cb_text}\n"
    f"Net 10y CO2 saved: {co2_saved:,.0f} kg",
    transform=ax.transAxes, fontsize=9, fontweight="bold", va="top",
    bbox=dict(boxstyle="round,pad=0.4", facecolor="#C8E6C9", edgecolor="#2E7D32"))

plt.suptitle("Heavy Truck Electrification – Financial Gap, Carbon Payoff",
             fontsize=14, fontweight="bold", y=1.02)
plt.tight_layout()
plt.show()

print(f"\n--- TRUCK 10-YEAR ANALYSIS ---")
print(f" Diesel TCO : {truck_diesel[-1]:,.0f} CHF")
print(f" EV TCO      : {truck_ev[-1]:,.0f} CHF")
print(f" Delta       : {truck_diesel[-1] - truck_ev[-1]:,.0f} CHF (negative = EV more expensive)")
print(f" Battery debt: {battery_debt:,.0f} kg CO2 at year 0")
print(f" Net CO2 saved over 10y: {co2_saved:,.0f} kg")
print(f" Carbon payback year: {truck_cb}")

```



```

--- TRUCK 10-YEAR ANALYSIS ---
Diesel TCO : 144,616 CHF
EV TCO      : 382,669 CHF
Delta       : -238,053 CHF (negative = EV more expensive)
Battery debt: 23,520 kg CO2 at year 0
Net CO2 saved over 10y: 37,881 kg
Carbon payback year: 4

```

```

# =====
# NPV-DISCOUNTED TCO
# =====

def plot_npv_breakeven():
    fig, axes = plt.subplots(1, 2, figsize=(15, 6))
    fig.patch.set_facecolor("#F8F9FA")

    horizon = PARAMS["horizon_years"]

```

```

years = list(range(horizon + 1))

for ax, (name, vdata) in zip(axes, VEHICLES.items()):
    diesel_und = compute_tco_curve(vdata, "diesel", PARAMS)
    ev_und = compute_tco_curve(vdata, "ev", PARAMS)
    diesel_npv = compute_npv_tco_curve(vdata, "diesel", PARAMS)
    ev_npv = compute_npv_tco_curve(vdata, "ev", PARAMS)

    ax.plot(years, diesel_und, "--", color="#E53935", alpha=0.45, label="Diesel (undiscounted)")
    ax.plot(years, ev_und, "--", color="#43A047", alpha=0.45, label="EV (undiscounted)")
    ax.plot(years, diesel_npv, "-o", color="#E53935", linewidth=2, markersize=5, label="Diesel (NPV @ 5%)")
    ax.plot(years, ev_npv, "-o", color="#43A047", linewidth=2, markersize=5, label="EV (NPV @ 5%)")

    View
    be_und = find_breakeven_year(diesel_und, ev_und)
    be_npv = find_breakeven_year(diesel_npv, ev_npv)

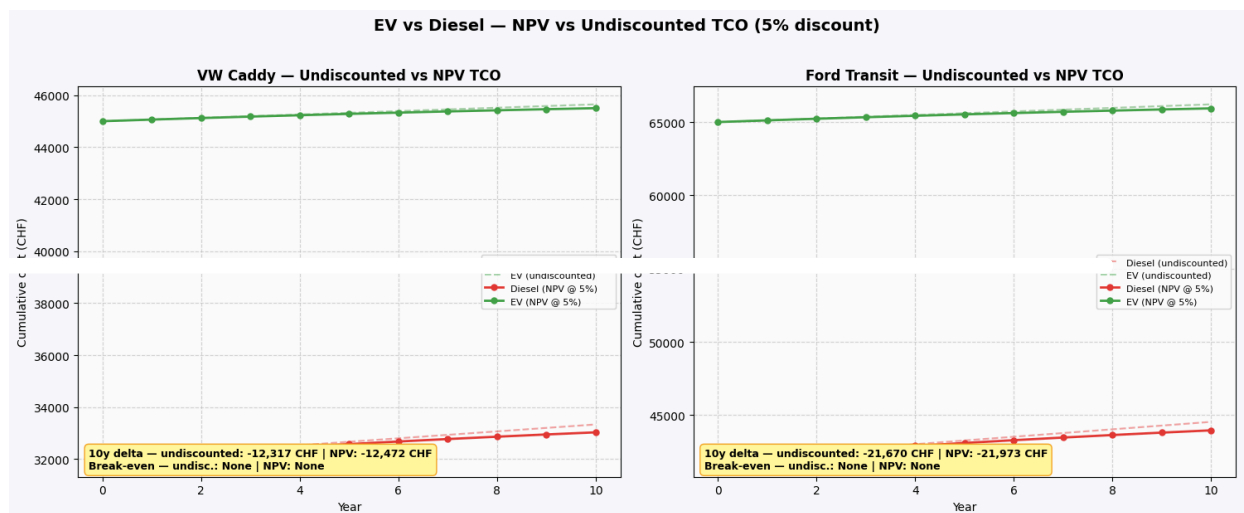
    delta_und = diesel_und[-1] - ev_und[-1]
    delta_npv = diesel_npv[-1] - ev_npv[-1]
    ax.text(0.02, 0.02,
           f"10y delta – undiscounted: {delta_und:,.0f} CHF | NPV: {delta_npv:,.0f} CHF\n"
           f"Break-even – undisc.: {be_und} | NPV: {be_npv}",
           transform=ax.transAxes, fontsize=9, fontweight="bold",
           bbox=dict(boxstyle="round,pad=0.4", facecolor="#FFF59D", edgecolor="#F9A825"))

    ax.set_title(f"{name} – Undiscounted vs NPV TCO", fontsize=12, fontweight="bold")
    ax.set_xlabel("Year")
    ax.set_ylabel("Cumulative cost (CHF)")
    ax.legend(fontsize=8)
    ax.grid(linestyle="--", alpha=0.5)
    ax.set_facecolor("#FAFAFA")

plt.suptitle("EV vs Diesel – NPV vs Undiscounted TCO (5% discount)",
            fontsize=14, fontweight="bold", y=1.02)
plt.tight_layout()
plt.show()

```

```
plot_npv_breakeven()
```



```

# =====
# ANNUAL MILEAGE SENSITIVITY
# =====

```

```

def km_sensitivity_table(vdata):
    km_range = [1_000, 2_500, 5_000, 7_500, 10_000, 15_000, 20_000, 25_000, 30_000]
    rows = []
    for km in km_range:
        d = compute_tco_curve(vdata, "diesel", PARAMS, override_km_year=km)
        e = compute_tco_curve(vdata, "ev", PARAMS, override_km_year=km)

```

```

    be = find_breakeven_year(d, e)
    rows.append({
        "annual_km": km,
        "diesel_10y_chf": round(d[-1]),
        "ev_10y_chf": round(e[-1]),
        "delta_10y_chf": round(d[-1] - e[-1]),
        "breakeven_year": be if be is not None else "-",
    })
return rows

def plot_km_sensitivity(url_distances):
    fig, axes = plt.subplots(1, 2, figsize=(15, 6))
    fig.patch.set_facecolor("#F8F9FA")
    # https://raw.githubusercontent.com/amathie...

    for ax, (name, vdata) in zip(axes, VEHICLES.items()):
        rows = km_sensitivity_table(vdata)
        kms = [r["annual_km"] for r in rows]
        deltas = [r["delta_10y_chf"] for r in rows]
        bes = [r["breakeven_year"] if isinstance(r["breakeven_year"], int) else None for r in rows]

        ax2 = ax.twinx()
        bars = ax.bar(kms, deltas, width=1500,
                      color=["#C62828" if d < 0 else "#2E7D32" for d in deltas],
                      alpha=0.7, edgecolor="black")
        ax.axhline(0, color="black", linewidth=0.8)
        ax.set_xlabel("Annual mileage (km)")
        ax.set_ylabel("10y delta CHF (Diesel - EV) - positive = EV cheaper")
        ax.set_title(f"{name} - Mileage Sensitivity", fontsize=12, fontweight="bold")
        ax.grid(axis="y", linestyle="--", alpha=0.4)
        ax.set_facecolor("#FAFAFA")

        # Plot break-even year on secondary axis
        be_kms = [k for k, b in zip(kms, bes) if b is not None]
        be_years = [b for b in bes if b is not None]
        if be_years:
            ax2.plot(be_kms, be_years, "-o", color="#1565C0", linewidth=2, markersize=7, label="Brea
            ax2.set_ylabel("Break-even year", color="#1565C0")
            ax2.tick_params(axis="y", labelcolor="#1565C0")
            ax2.set_ylim(0, PARAMS["horizon_years"] + 1)
            ax2.legend(loc="upper right")

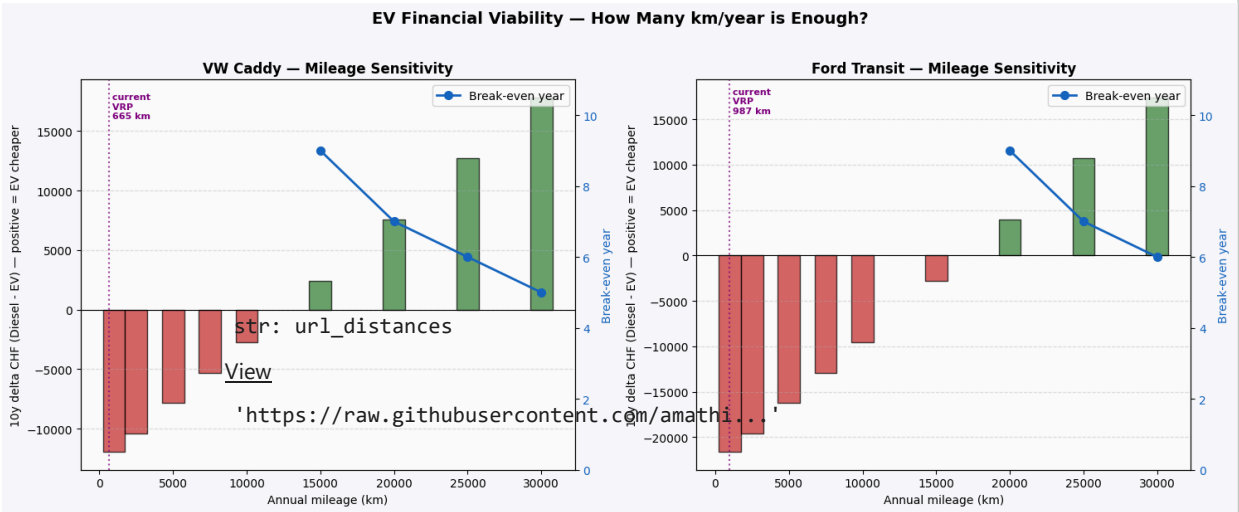
        # Mark current VRP mileage
        current = vdata["weekly_km"] * WEEKS_PER_YEAR
        ax.axvline(current, color="purple", linestyle=":", alpha=0.7)
        ax.text(current, max(deltas) * 0.9 if max(deltas) > 0 else min(deltas) * 0.9,
                f" current\n VRP\n {current:.0f} km",
                color="purple", fontsize=8, fontweight="bold")

    plt.suptitle("EV Financial Viability - How Many km/year is Enough?",
                 fontsize=14, fontweight="bold", y=1.02)
    plt.tight_layout()
    plt.show()

# Print tables
for name, vdata in VEHICLES.items():
    print(f"\n--- {name} ---")
    rows = km_sensitivity_table(vdata)
    print(f"{'km/year':>8} {'Diesel 10y':>13} {'EV 10y':>13} {'Delta':>13} {'BE year':>10}")
    for r in rows:
        print(f"{r['annual_km']:>8}, {r['diesel_10y_chf']:>13}, {r['ev_10y_chf']:>13}, "
              f"{r['delta_10y_chf']:>13}, {str(r['breakeven_year']):>10}")

plot_km_sensitivity()

```



--- VW Caddy ---

km/year	Diesel 10y	EV 10y	Delta	BE year
1,000	34,002	45,975	-11,972	—
2,500	37,006	47,438	-10,431	—
5,000	42,012	49,875	-7,862	—
7,500	47,019	52,312	-5,294	—
10,000	52,025	54,750	-2,725	—
15,000	62,038	59,625	2,412	9
20,000	72,050	64,500	7,550	7
25,000	82,062	69,375	12,688	6
30,000	92,075	74,250	17,825	5

--- Ford Transit ---

km/year	Diesel 10y	EV 10y	Delta	BE year
1,000	44,572	66,225	-21,652	—
2,500	48,431	68,062	-19,631	—
5,000	54,862	71,125	-16,262	—
7,500	61,294	74,188	-12,894	—
10,000	67,725	77,250	-9,525	—
15,000	80,588	83,375	-2,788	—
20,000	93,450	89,500	3,950	9
25,000	106,312	95,625	10,688	7

=====
DIESEL x ELECTRICITY PRICE HEATMAP
